

Integration of Mixed-Criticality into Dynamic Task Migration for Real-Time Embedded Systems

Octavio Ivan Delgadillo Ruiz

Vollständiger Abdruck der von der Fakultät für Informatik der Technischen Universität München zur Erlangung des akademischen Grades eines

PhD (check specific degree))

genehmigten Dissertation.

Vorsitzender:

Prof. Dr. Uwe Baumgarten

Prüfende der Dissertation:

1. Prof. Dr.-Ing. Vorname Nachname
2. Prof. Dr.-Ing. Vorname Nachname

Die Dissertation wurde am 01.01.2016 bei der Technischen Universität München eingereicht und durch die Fakultät für Informatik am 10.05.2016 angenommen.

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Zusammenfassung

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Contents

Abstract	iii
Zusammenfassung	v
Contents	vii
List of Figures	xi
List of Tables	xiii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Research Methodology	4
1.5 Contributions	5
1.6 Dissertation Structure	6
2 Background and Fundamentals	7
2.1 Real-Time Systems	7
2.1.1 Scheduling Theory	8
2.1.2 Real-Time Operating Systems	9
2.2 Mixed-Criticality Systems	10
2.2.1 Criticality Levels and Standards	10
2.2.2 Scheduling and Resource Management for MCS	11
2.3 Distributed Real-Time Systems	12
2.3.1 End-to-End Timing Analysis	12
2.3.2 Heterogeneous and Edge Computing Platforms	13
2.3.3 Task Migration and Runtime Adaptation	13
2.4 Networks and Communication in Real-Time Systems	14
2.4.1 Time-Sensitive Networking	14
2.4.2 Service-Oriented Communication and Middleware	15
2.5 Summary	15

CONTENTS

3	Related Work and State of the Art	17
3.1	Scheduling and Resource Management in Real-Time Systems	17
3.1.1	Multiprocessor and Distributed Scheduling	18
3.2	Mixed-Criticality Systems	18
3.2.1	Adaptive and Multicore Mixed-Criticality Scheduling	19
3.2.2	Mixed-Criticality in Distributed and Fault-Tolerant Systems . . .	19
3.3	Distributed Real-Time Systems and Automotive Architectures	20
3.3.1	Automotive Architecture Evolution	20
3.3.2	Time-Sensitive Networking	21
3.4	Task Offloading and Orchestration in IoT and Edge Systems	21
3.5	Task Migration in Real-Time Systems	22
3.5.1	Migration Overhead and Schedulability	23
3.5.2	Fault-Tolerant Migration in Embedded Systems	23
3.5.3	Runtime Migration in Distributed Real-Time Systems	23
3.6	Machine Learning for Scheduling and Deployment	24
3.7	Prior Work at the Chair	25
3.8	Research Gap Summary	26
4	Approach	29
4.1	Breathability	29
4.2	Use Cases	30
4.3	System Model	31
4.3.1	Nodes and Sub-Nodes	31
4.3.2	Tasks, Jobs and Criticality	33
4.3.3	Criticality Levels and Node Ceilings	34
4.3.4	Migration Events	34
4.3.5	Time	34
4.4	Architectural Overview	38
4.5	Criticality-Stratified Migration	40
4.6	Design Decisions	41
4.7	Scope of the Safety Argument	42
4.8	Design Realisation and Status	43
5	Base Operating System: Axcan OS	45
5.1	Design Rationale and Relationship to Existing Software	45
5.2	Architecture	46
5.2.1	Multi-Core Nodes	48
5.3	Mixed-Criticality Scheduler	49
5.3.1	Baseline: EDF under ESFREE	49
5.3.2	EDF with Virtual Deadlines	49
5.3.3	Realisation Variants	50
5.3.4	Admission Control	50

5.3.5	Timing Basis	51
5.4	Dynamic Task Deployment	51
5.4.1	Multi-Core Deployment	51
5.4.2	Inter-Core Communication	52
5.5	Cross-Platform Task Loading	52
5.5.1	Hardware Abstraction Layer	53
5.6	The Guest Task Model	53
5.6.1	The Wrapper Task	53
5.6.2	Compilation Requirements	54
5.6.3	Execution Model and Checkpoints	54
5.7	Memory Management and Isolation	55
5.8	Failure Handling	55
5.9	Resource Footprint	55
5.10	Summary	56
6	Task Migration Implementation	59
6.1	Statefulness and Checkpoint Mechanism	59
6.2	High Criticality Strategy	60
6.3	Low Criticality Strategy	61
6.4	MEC: Virtual Devices	61
7	System Integration	63
7.1	Real-Time Constraints in the Distributed System	63
7.2	Adaption of the ML algorithm	63
7.3	Virtual and Physical Devices	63
8	Results and Evaluation	65
8.1	Test Setup	65
8.2	Discussion	65
9	Conclusion & Outlook	67
	Bibliography	69
A	Additional Stuff	75

List of Figures

2.1	Example of Schedules Generated Using Different Scheduling Policies	8
4.1	General system overview: the global task queue is partitioned onto physical and virtual nodes, each running a mixed-criticality scheduler, with migration events transferring tasks between them.	39
5.1	Axcan OS architecture. The FreeRTOS kernel and hardware abstraction layer are used as provided; the remaining components are contributions of this work.	47
5.2	AMP deployment of Axcan OS on the quad-core ARM Cortex-A53 of the target hardware. Each core runs an independent instance and constitutes a node.	48
5.3	Differences between the Axcan OS instances on the master core and on a slave core.	49
5.4	A single precompiled application executing across a bare-metal embedded platform, a POSIX-hosted platform, and a containerised virtual node, all sharing the ARM AArch64 architecture.	52
5.5	Guest application structure. Left: the minimal form, in which state is restored at the start of a job and stored at its end. Right: the extended form with an intra-job checkpoint and jump tag, permitting resumption from an internal point.	55

List of Tables

4.1	Summary of notation.	35
4.2	Traceability from research questions and requirements to mechanisms and chapters.	43
4.3	Realisation status of the mechanisms at the time of writing.	44
5.1	Failure modes introduced by dynamic loading and their handling. . . .	56
5.2	Static memory footprint of Axcan OS relative to the base kernel, ARM Cortex-A53 port.	56

1 Introduction

1.1 Motivation

The automotive industry is undergoing a fundamental transformation driven by the growing role of software and electronics in vehicle design and innovation. Industry analyses consistently report that the majority of automotive innovation today originates in electronic systems rather than in mechanical or materials engineering [HC16], and that spending on automotive software and electronics is projected to account for nearly half of the total vehicle bill of materials in premium vehicles by the end of this decade [KSK18]. Vehicle electrification, autonomous driving, advanced driver assistance systems (ADAS), and vehicle-to-everything (V2X) connectivity are only some of the domains where onboard computing has become central to product differentiation and safety.

A modern high-end vehicle contains upward of 100 Electronic Control Units (ECUs) connected by several kilometers of wiring harness, contributing significantly to vehicle weight and manufacturing cost [Vip18; Bau+18]. Historically, ECUs have been designed with a single-purpose philosophy: each unit is dedicated to one functional domain in order to avoid interference between tasks and to simplify safety certification [Vip+14; Vip18]. While this approach provides clear isolation and predictable behavior, it has well-known structural shortcomings. Many tasks are only active during specific and rare operating conditions, leading to chronic underutilization of processing resources across the fleet. A failure of any single ECU with no fallback mechanism can also compromise the functions it hosts. The combined effect is a system that is simultaneously over-provisioned in hardware and fragile in fault tolerance.

These inefficiencies have motivated a broader industry transition toward *ECU consolidation*: redistributing workloads from many single-purpose units to a smaller number of high-performance, multi-purpose computing platforms [Bur+18]. The emerging software-defined vehicle (SDV) concept extends this further, treating vehicle functions as software services that can be deployed, updated, and redistributed across a flexible hardware architecture at runtime [Ela+24]. Under this paradigm, the static binding between a function and a specific ECU is relaxed, and the ability to dynamically distribute and redistribute tasks at runtime becomes a key enabler of both resource efficiency and fault tolerance.

Achieving this vision, however, requires solving a problem that sits at the intersection of two fields that have historically developed largely independently: *mixed-criticality systems* and *real-time task migration*. This tension is the central intellectual challenge of

this dissertation, and it is worth stating explicitly from the outset.

Mixed-criticality (MC) systems research is fundamentally concerned with *predictability and static assurance*: given a set of tasks with heterogeneous safety requirements, expressed through standards such as ISO 26262 Automotive Safety Integrity Levels (ASILs) [Iso], how can a platform guarantee that high-criticality tasks always meet their deadlines, even under worst-case conditions, while also providing useful service to lower-criticality workloads? The entire analytical apparatus of mixed-criticality scheduling, from Vestal’s foundational model [Ves07] to more recent adaptive approaches [BD17], is built on the assumption that task parameters are known and fixed at design time, and that the platform topology does not change during execution.

Dynamic task migration, by contrast, is inherently disruptive to this assumption. Migration transfers an executing task from one node to another, necessarily interrupting its execution, serializing its state, transmitting that state over a network, and resuming execution in a different environment. Each of these steps introduces latency, consumes resources on both the source and destination node, and temporarily violates the static assumptions on which schedulability guarantees are built. In a system with hard real-time constraints, where a braking control function failing its deadline can have fatal consequences, this disruption must be managed with extreme care.

Reconciling these two requirements is non-trivial. It is not sufficient to apply existing MC scheduling on each node independently and treat migration as a separate layer: the act of migration changes the task set on each node, triggering mode transitions, modifying resource availability, and potentially invalidating schedulability guarantees that were valid before the migration event. Conversely, existing real-time migration approaches do not account for criticality-differentiated treatment of tasks, providing no mechanism to prioritize the migration of safety-critical functions over best-effort ones when resources are constrained. The intersection of these two fields, particularly in the context of ECU consolidation for automotive systems, represents a largely unexplored area in the published literature, and constitutes the primary motivation for this dissertation.

1.2 Problem Statement

Prior research at the Chair of Operating Systems of the Technical University of Munich (TUM), conducted within projects including KIA4SM, MaLSAMi, and HaCRoM, has established foundational approaches to task migration in real-time distributed systems. This includes checkpoint-based migration execution mechanisms and machine-learning-based migration planning tools capable of runtime schedulability prediction [Tai19; Utz19; Bli19]. Some of these contributions form the direct technical basis on which this dissertation builds. However, they share some critical limitations: they operate at a single, undifferentiated criticality level. All tasks are treated equivalently with respect to scheduling priority and migration policy, and no mechanism exists

to guarantee that high-criticality tasks are privileged during resource contention or migration events. Furthermore, so far no actual migration mechanism has been implemented that is compatible with execution on a real-time operating system with minimal overhead and fulfilling criticality constraints.

These limitations become untenable in the context of ECU consolidation for safety-critical automotive systems. When an ASIL-D braking function and a non-safety-relevant navigation application share the same computing platform and the same migration infrastructure, they must be treated fundamentally differently. ISO 26262 mandates that the behavior of safety-critical functions be guaranteed and verifiable under all relevant operating conditions [Iso]; a migration mechanism that might delay or interrupt such a function in the same way as a low-criticality task is not compliant with this requirement.

Four concrete open problems define the scope of this dissertation. First, the base execution platform must be extended to support dynamic loading of executables with a corresponding checkpoint-restore mechanism. Second, the execution platform must also be extended to support mixed-criticality scheduling, providing the runtime separation of concerns required for correct MC behavior before any migration mechanism is layered on top. Third, differentiated migration strategies must be designed for tasks at different criticality levels, each with appropriate timing guarantees, state preservation mechanisms, and resource overhead characteristics. The fundamental challenge here is that high-criticality migration must be fast, bounded, and formally analyzable, while low-criticality migration may trade immediacy for efficiency. Fourth, the interaction between migration events and the MC scheduler on each node must be carefully accounted for: a task arriving at a destination ECU mid-execution has different schedulability implications than a task present since system startup, and the platform design must handle this correctly.

A secondary problem concerns the role of edge computing resources in this architecture. Vehicular edge cloud (VEC) and Mobile Edge Computing (MEC) infrastructures place compute capacity in proximity to the vehicle, offering supplementary processing resources with latencies that may be acceptable for non-safety-critical workloads. Whether and how such resources can be incorporated as a third tier in a criticality-stratified migration hierarchy, and what formal conditions bound their safe use, is an exploratory research question this dissertation investigates.

1.3 Research Questions

Based on the problem statement above, this dissertation is guided by the following research questions:

- RQ1.** To what extent is task migration feasible in a distributed real-time embedded system while respecting real-time constraints, and under what conditions can timing guarantees be preserved across a migration event?

- RQ2.** How should tasks be prioritized and scheduled based on their criticality level in a system that must adapt dynamically to hardware failures and varying task load, and how can mixed-criticality scheduling be integrated with a runtime migration execution layer?
- RQ3.** What migration strategies, at both the operating system and communication levels, are most suitable for high- and low-criticality tasks, considering the trade-off between minimizing migration latency and optimizing resource utilization?
- RQ4.** Under what conditions is it feasible to offload tasks to virtual edge resources as part of a criticality-stratified migration hierarchy, and what are the implications with respect to latency, schedulability, safety assurance, and resource efficiency?

1.4 Research Methodology

This dissertation follows a design-science research methodology, proceeding through three stages: system design, prototypical implementation on real hardware, and empirical evaluation.

The first stage is *system design*. The architecture of the mixed-criticality base platform, the guest task model, and the differentiated migration strategies are developed through iterative design informed by the requirements of the target domain. Design decisions are driven by the constraints of embedded ECU-class hardware (limited memory, deterministic scheduling requirements, and the isolation demands imposed by ISO 26262) rather than by theoretical optimality criteria alone.

The second stage is *prototypical implementation*. The designed platform is realized as a working system on embedded hardware, integrating the Axcan OS mixed-criticality scheduler, an ELF-based guest task loading mechanism, checkpoint and state transfer support, and a communication infrastructure based on PTP time synchronization and SOME/IP. This prototype serves as the primary artifact through which the design is evaluated.

The third stage is *empirical evaluation*. The implemented system is evaluated through proof-of-concept demonstrations and direct measurement on the target hardware. Initial results characterize the task loading latency achievable on embedded nodes and on virtualized edge nodes respectively, establishing an empirical baseline for assessing migration feasibility across criticality tiers. Further timing and schedulability measurements are planned as the platform matures. The emphasis throughout is on grounding claims in observed system behavior on real hardware, rather than in simulation or purely analytical bounds.

1.5 Contributions

This dissertation makes the following original contributions to the state of the art in mixed-criticality real-time systems and distributed embedded computing:

1. **Mixed-criticality integration into a distributed real-time embedded platform.** The FreeRTOS-based ESFREE platform is extended to support mixed-criticality scheduling with an Asymmetric Multiprocessing (AMP) execution model on multicore ECU hardware. This provides the foundational runtime separation of criticality levels required for all subsequent contributions.
2. **Guest task architecture and ELF-based migration support.** A portable guest task architecture is defined in the form of a C library and a simplified linker script, enabling tasks to be compiled as self-describing ELF binaries that can be loaded and relocated at runtime on a target ECU. The architecture integrates checkpoint support and PTP-synchronized timing to preserve task progress and timing context across migration events.
3. **Criticality-differentiated migration strategies.** Three migration strategies are defined and implemented, each targeting a distinct criticality tier. The high-criticality (HI) strategy provides a fast, formally bounded migration path with timing guarantees sufficient for ASIL-class tasks. The low-criticality (LO) strategy employs a checkpoint-restore mechanism that prioritizes continuity of progress while minimizing interference with co-hosted high-criticality tasks. The Virtual ECU strategy defines the conditions and mechanisms under which non-safety-critical tasks may be offloaded to edge computing resources, and characterizes the associated latency and reliability constraints.
4. **Empirical characterization of task loading latency across execution tiers.** Initial measurements on the target hardware establish the task loading latency achievable by the Axcan OS ELF loading mechanism on embedded nodes, and the additional overhead introduced by containerization on virtualized edge nodes. These results provide a concrete empirical baseline for assessing the feasibility and overhead of migration across the different criticality tiers, and motivate the design choices made in the differentiated migration strategies.
5. **Mixed-criticality-aware adaptation of an ML-based migration planning toolchain.** The machine-learning-based schedulability prediction tools developed in prior theses [Tai19; Utz19; Bli19] are adapted to embedded ECU-class hardware and extended to incorporate mixed-criticality task parameters, enabling MC-aware runtime migration planning.

1.6 Dissertation Structure

The remainder of this dissertation is organized as follows.

**** TODO: Update this with correct references to chapters, keeping this structure as placeholder

Chapter 2 introduces the fundamental concepts in real-time systems, mixed-criticality scheduling, distributed embedded architectures, task migration, and communication technologies for cyber-physical systems. Chapter 3 surveys the state of the art relevant to each research question and identifies the open problems addressed by this work. Chapter 4 presents the system architecture and design, including the mixed-criticality base platform, guest task model, and migration infrastructure. Chapter 5 describes the three criticality-differentiated migration strategies in detail, including their design rationale, implementation, and initial empirical characterization. Chapter 6 presents the empirical evaluation, including schedulability analysis results, migration latency measurements, and proof-of-concept demonstrations on the target hardware platform. Chapter 7 concludes the dissertation, summarizes the contributions with respect to the research questions, and identifies directions for future work.

2 Background and Fundamentals

This chapter introduces the fundamental concepts required for understanding the research presented in this dissertation. First, the foundations of real-time systems and scheduling theory are discussed, including timing constraints, scheduling algorithms, and real-time operating systems. Afterwards, the concept of mixed-criticality systems is introduced together with the challenges associated with resource sharing and adaptability in modern embedded platforms. Subsequently, distributed real-time systems and modern automotive computing architectures are analyzed, including heterogeneous systems, edge computing, and task offloading concepts. Finally, the chapter presents the networking technologies and communication mechanisms commonly used in distributed cyber-physical systems with real-time requirements.

2.1 Real-Time Systems

Real-time systems are computing systems in which correctness depends not only on the logical correctness of the produced results, but also on the time at which these results are generated [But11]. In such systems, timing constraints are treated as first-class requirements and must be satisfied in order to guarantee correct system behavior. Real-time systems are commonly classified into hard real-time, firm real-time, and soft real-time systems depending on the consequences of missing timing deadlines. In hard real-time systems, missing a deadline may lead to catastrophic failures, making deterministic timing guarantees essential. Firm real-time systems occupy an intermediate position: a missed deadline renders the result valueless, but unlike hard real-time systems, individual misses do not necessarily trigger a system failure. Soft real-time systems, on the other hand, can tolerate occasional deadline misses with graceful degradation of quality of service.

Each task in a real-time system is typically characterized by a set of parameters: its release time (or arrival time), an execution time, a relative deadline, and—for periodic tasks—a period. The Worst-Case Execution Time (WCET) is a particularly critical parameter, as it represents an upper bound on the time a task may require under any input and execution condition. Deriving tight and safe WCET estimates is a research challenge in its own right, with techniques ranging from static analysis [Wil+08] to measurement-based approaches [Eng+03]. The inherent trade-off between safety (overestimation) and efficiency (tight bounds) is especially pronounced in heterogeneous and many-core platforms, where cache effects, pipeline speculation, and memory

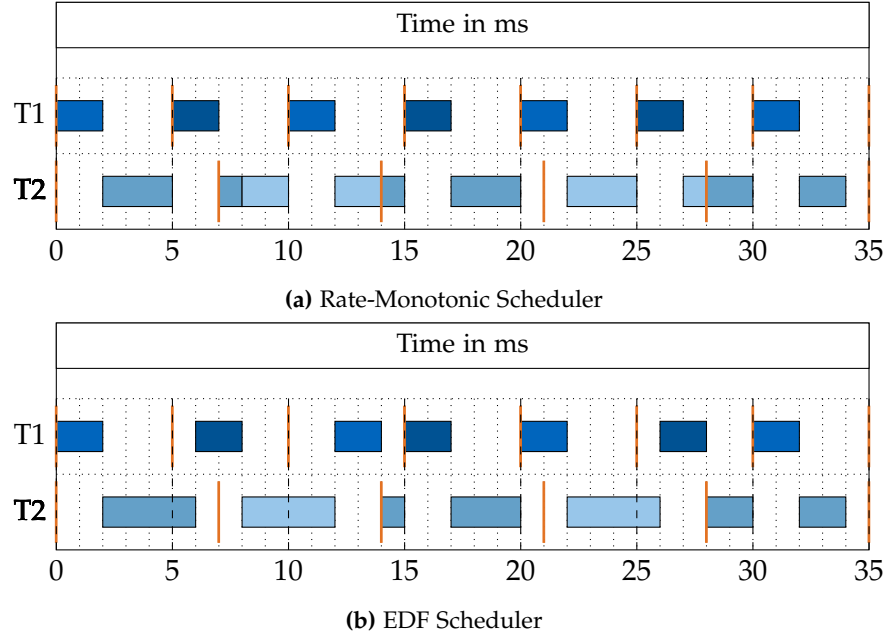


Figure 2.1: Example of Schedules Generated Using Different Scheduling Policies

contention increase analysis complexity.

Modern cyber-physical systems such as automotive platforms, industrial controllers, robotics systems, and autonomous vehicles increasingly depend on strict timing guarantees while simultaneously requiring higher computational flexibility and adaptability [Bau+18; Ela+24]. This combination introduces significant challenges for system design and resource management.

2.1.1 Scheduling Theory

Scheduling is one of the core concepts in real-time systems and refers to the process of determining the execution order of tasks while satisfying timing constraints and resource limitations. Classical real-time scheduling approaches include fixed-priority scheduling algorithms such as Rate Monotonic (RM), as well as dynamic-priority approaches such as Earliest-Deadline-First (EDF).

Liu and Layland [LL73] introduced one of the foundational scheduling models for periodic real-time tasks, establishing schedulability bounds for rate-monotonic (RM) and earliest-deadline-first (EDF) scheduling. EDF scheduling is particularly relevant due to its optimality properties for uniprocessor systems under specific assumptions: it is known to be optimal in the sense that if any algorithm can schedule a task set on a single processor, EDF can also schedule it. Fixed-priority schemes such as RM, while not theoretically optimal, offer simpler analysis and more predictable behavior in

the presence of task interactions, making them dominant in practice. A comparison between schedules generated with both RM and EDF policies is shown in Fig. 2.1, where we can notice that in many cases, the dynamic policy of EDF is able to find a schedule without missing deadlines, while RM misses deadlines for T2 in job 1.

Schedulability analysis is the formal process of determining whether a task set will always meet its deadlines under a given scheduling policy. Response Time Analysis (RTA) for fixed-priority systems [JP86] and utilization-based bounds for EDF [LL73] are well-established analytical tools. However, extending such analyses to account for blocking from shared resources, interrupt handling, and inter-task communication requires additional models such as the Sporadic Server or the Stack Resource Policy [Bak91].

As modern embedded systems evolved towards multicore and heterogeneous architectures, scheduling approaches also became more complex. Multiprocessor scheduling introduces additional challenges including task migration overhead, synchronization, resource sharing, cache interference, and communication latency [MSD10]. Broadly, multiprocessor scheduling strategies are classified into partitioned, global, and semi-partitioned (or clustered) approaches. Partitioned scheduling assigns each task permanently to one processor and applies uniprocessor analysis independently, at the cost of possible inefficiency due to bin-packing constraints. Global scheduling allows tasks to migrate freely across processors, improving utilization but introducing runtime migration overhead and making formal analysis significantly harder. Semi-partitioned and cluster-based approaches attempt to balance these trade-offs [DB11].

2.1.2 Real-Time Operating Systems

Real-Time Operating Systems (RTOSs) provide the software infrastructure required for executing time-critical tasks while maintaining deterministic system behavior. Compared to general-purpose operating systems, RTOSs prioritize predictability, bounded latency, and deterministic scheduling behavior over throughput optimization.

Common RTOS functionalities include task scheduling, inter-process communication, synchronization primitives, interrupt handling, and memory management. Depending on the system requirements, RTOSs may support static scheduling, dynamic scheduling, partitioning, or mixed-criticality execution models. Well-known examples include OSEK/VDX [Ose] and AUTOSAR Classic Platform, which are widely deployed in automotive embedded systems, as well as more general-purpose RTOSs such as FreeRTOS and VxWorks.

The AUTOSAR Adaptive Platform [Aut] represents a more recent development targeted at high-performance automotive computing nodes. Unlike the Classic Platform, which is oriented towards microcontrollers with static task configurations, the Adaptive Platform supports dynamic application deployment, service-oriented communication, and POSIX-compliant execution environments, all while maintaining defined timing and isolation guarantees. This architectural shift is particularly relevant for the integra-

tion of mixed-criticality workloads and dynamic task management in modern vehicle platforms.

In distributed and heterogeneous systems, RTOSs additionally play an important role in enabling communication, coordination, and resource sharing between different hardware nodes. The ability to support runtime adaptation and dynamic task migration increasingly represents an important requirement for modern distributed embedded systems. Hypervisor-based virtualization is increasingly used to consolidate multiple RTOS instances or mixed-criticality partitions on a single hardware platform [Hei20], providing spatial and temporal isolation with lower hardware costs.

2.2 Mixed-Criticality Systems

Mixed-criticality systems (MCS) are systems in which applications with different levels of criticality coexist on shared hardware resources [Ves07; BD17]. Such systems are especially relevant in domains including automotive systems, avionics, robotics, and industrial automation, where safety-critical and non-safety-critical applications execute simultaneously within the same platform.

Traditionally, critical systems relied on strong spatial and temporal partitioning mechanisms in order to isolate workloads with different certification requirements. While this approach improves safety and predictability, it may also lead to low resource utilization and reduced flexibility.

Vestal [Ves07] introduced one of the most influential mixed-criticality models, where tasks are associated with different Worst-Case Execution Time (WCET) estimations depending on their assurance level. Specifically, a high-criticality task carries a pessimistic (safe) WCET estimate for use under high-assurance conditions, and a more optimistic estimate for use when the system operates in a nominal low-load mode. Since then, multiple scheduling approaches have been proposed to support dynamic adaptation and graceful degradation under overload conditions. The mixed-criticality model of Baruah et al. [Bar+11] extended Vestal's original formulation to a formal schedulability framework under EDF, providing a foundation for a significant body of subsequent research.

2.2.1 Criticality Levels and Standards

In safety-critical domains, criticality levels are often defined by domain-specific standards. In avionics, the DO-178C standard defines Design Assurance Levels (DALs) A through E for software, where DAL A represents the highest assurance requirement for functions whose failure could cause catastrophic consequences [Do1]. The corresponding hardware standard DO-254 applies analogous levels to electronic hardware design. In automotive systems, ISO 26262 defines Automotive Safety Integrity Levels (ASILs) ranging from ASIL A to ASIL D, with ASIL D representing the most stringent require-

ments for functions whose failure could lead to severe injuries or fatalities [Iso]. For general functional safety across industrial domains, IEC 61508 defines Safety Integrity Levels (SILs) 1 through 4.

These standards impose requirements not only on the software algorithms themselves but also on the processes, tools, verification methods, and evidence used during development. When components of different criticality share hardware resources, demonstrating compliance becomes significantly more complex, as the behavior of lower-criticality components may potentially interfere with higher-criticality ones. Achieving compositionality, i.e. the ability to verify system properties from verified component properties, is a central challenge in the design of mixed-criticality platforms.

2.2.2 Scheduling and Resource Management for MCS

Modern mixed-criticality systems increasingly explore adaptive resource management techniques in order to improve efficiency while preserving timing guarantees [Sal+23]. In the standard mixed-criticality scheduling model, the system can transition between a low-criticality (LO) mode and a high-criticality (HI) mode. When the observed execution time of a high-criticality task exceeds its LO-mode WCET estimate, the system switches to HI mode, abandoning or reducing the service of low-criticality tasks in order to guarantee that all high-criticality deadlines are met.

Several extensions to this basic model have been proposed. The AMC (Adaptive Mixed Criticality) scheduler [BB13] uses per-task mode switching rather than a global transition, allowing finer-grained adaptation. Elastic scheduling approaches attempt to dynamically adjust task parameters (periods, execution budgets) in response to load changes while maintaining feasibility constraints. Research on *service degradation* policies addresses the question of how best to reduce the service of lower-criticality tasks during overload, recognizing that an abrupt shutdown of lower-criticality functions may itself introduce safety risks.

Recent research has also investigated stronger isolation mechanisms and heterogeneous execution models for mixed-criticality platforms [HGB+24; TV24]. Cache coloring and cache partitioning have emerged as important mechanisms for reducing cache interference between tasks of different criticality levels executing on shared-cache multicore platforms [Man+13]. Memory bandwidth regulation and the use of memory protection units (MPUs) or memory management units (MMUs) further contribute to spatial isolation guarantees.

In distributed cyber-physical systems such as modern automotive platforms, mixed-criticality introduces additional complexity due to communication delays, distributed resource sharing, synchronization constraints, and heterogeneous hardware architectures. These systems must simultaneously maintain timing guarantees, fault tolerance, and runtime adaptability across multiple interconnected nodes. The integration of mixed-criticality scheduling with distributed resource management and task migration constitutes a central open challenge addressed in this dissertation.

2.3 Distributed Real-Time Systems

Distributed real-time systems consist of multiple computing nodes that cooperate in order to execute time-critical tasks while communicating over a network infrastructure. Such systems are commonly used in domains including automotive electronics, industrial automation, aerospace systems, and large-scale cyber-physical systems. In contrast to uniprocessor or shared-memory multiprocessor systems, distributed systems must cope with the inherent uncertainty of network communication, including variable transmission delays, packet losses, and clock synchronization errors, all while satisfying end-to-end timing constraints.

Modern automotive systems increasingly evolve from domain-oriented architectures composed of numerous Electronic Control Units (ECUs) towards centralized and zonal architectures based on high-performance computing platforms [Ela+24; Ben+23]. This transition is motivated by the growing computational requirements of software-defined functionalities, autonomous driving systems, and advanced driver assistance systems (ADAS). A zonal architecture groups ECUs by physical location in the vehicle rather than by function, with high-bandwidth Ethernet backbones connecting zone controllers to central compute nodes. This consolidation reduces wiring complexity and weight while enabling more powerful centralized processing, but also concentrates risk and demands stronger mechanisms for isolation and fault containment.

Distributed architectures provide several advantages, including scalability, fault tolerance, workload distribution, and resource sharing. However, they also introduce challenges associated with synchronization, communication latency, scheduling coordination, and distributed resource management.

2.3.1 End-to-End Timing Analysis

A fundamental challenge in distributed real-time systems is the analysis of end-to-end latency for processing chains that span multiple nodes. A *cause-effect chain* (or task chain) is a sequence of tasks executing on potentially different nodes, where each task consumes the output of the previous one. The end-to-end latency of such a chain is determined not only by the execution times of individual tasks, but also by their activation patterns, scheduling decisions on each node, and the communication delays incurred at each link.

The problem of bounding end-to-end latency has been studied through several analytical frameworks. The work of Dürr et al. [Dür+19] provides a systematic characterization of end-to-end latency semantics, distinguishing between *reaction time* (the worst-case delay from an external event to the completion of the last task in the chain) and *data age* (the maximum staleness of data observed by the final consumer). These two metrics capture different aspects of real-time correctness relevant to different application domains; for example, control systems are typically more sensitive to data age, whereas event-triggered safety responses demand short reaction times.

2.3.2 Heterogeneous and Edge Computing Platforms

Modern distributed embedded systems frequently combine heterogeneous computing resources including CPUs, GPUs, FPGAs, and specialized accelerators. Such heterogeneous platforms can improve performance and energy efficiency but also increase system complexity and scheduling challenges. The diversity of instruction set architectures, memory hierarchies, and programming models across these processing elements means that tasks cannot in general be freely migrated without adaptation of the application binary or execution environment.

The emergence of edge and Multi-access Edge Computing (MEC) architectures additionally enables the distribution of tasks between embedded devices, edge servers, and cloud infrastructure [Mec]. This paradigm supports workload offloading, adaptive deployment, and resource elasticity while reducing latency compared to centralized cloud computing approaches. In latency-sensitive cyber-physical applications, edge computing plays a particularly important role by enabling local processing of sensor data with sub-millisecond response times that would be infeasible with round-trips to a remote cloud.

Task offloading and workload distribution mechanisms are closely related to orchestration concepts commonly used in cloud and Internet-of-Things (IoT) environments. In such systems, orchestration mechanisms coordinate deployment, resource allocation, monitoring, and execution management across multiple distributed nodes. Container-based virtualization and lightweight orchestration frameworks, originally developed for cloud-native applications, are increasingly being adapted for edge and embedded contexts [Mor17], though the real-time and safety guarantees required in cyber-physical systems impose constraints that are not natively addressed by mainstream cloud orchestration tools.

2.3.3 Task Migration and Runtime Adaptation

Task migration refers to the process of transferring a task or workload from one execution node to another during system operation. Migration mechanisms may be used for fault tolerance, load balancing, resource optimization, energy management, or adaptive scheduling purposes.

Existing migration approaches range from static failover mechanisms to fully dynamic runtime migration strategies. However, implementing migration in real-time systems introduces several challenges, including state transfer, synchronization overhead, migration latency, and timing predictability [CL12; MSD10]. The migration process itself can be decomposed into several phases: (i) the *pre-migration* phase, in which the source node prepares and serializes the task state; (ii) the *transfer* phase, during which state data is transmitted to the destination node over the interconnect; and (iii) the *post-migration* phase, in which the destination node initializes execution from the transferred state. Each of these phases introduces latency that must be accounted

for in the schedulability analysis of the migrating task as well as of any tasks that share resources with it.

Checkpoint-based migration approaches periodically save a task's execution state to stable storage or shared memory, enabling resumption from the most recent checkpoint after a failure or migration event [Eln+02]. While this reduces the cost of the transfer phase by transmitting incremental state differences, it introduces regular runtime overhead for checkpointing and may not be compatible with hard real-time timing guarantees unless the checkpointing overhead is explicitly bounded and included in schedulability analysis.

In heterogeneous distributed systems, migration becomes even more complex due to architectural differences between hardware platforms, communication constraints, and execution environment compatibility. Binary incompatibility between different instruction set architectures requires either maintaining multiple compiled variants of each migratable task, recompilation at migration time, or the use of intermediate representations or virtual machines that abstract the underlying hardware. The overhead introduced by such compatibility layers must be carefully evaluated against the latency and correctness requirements of the target application.

2.4 Networks and Communication in Real-Time Systems

Communication networks represent a critical component of distributed cyber-physical systems. In real-time systems, communication mechanisms must provide deterministic behavior, bounded latency, synchronization support, and fault tolerance.

Traditional automotive systems relied heavily on fieldbus technologies such as Controller Area Network (CAN), FlexRay, and Local Interconnect Network (LIN). CAN provides a broadcast, message-priority-based medium access scheme with well-understood worst-case latency bounds, but its bandwidth is limited to 1 Mbit/s (or up to 8 Mbit/s with CAN FD). FlexRay offers higher bandwidth and a time-triggered communication model that provides stronger temporal determinism, but at higher cost and complexity. LIN is used for low-bandwidth sensor and actuator networks where cost is paramount. These technologies collectively supported the domain-oriented ECU architectures of previous vehicle generations.

2.4.1 Time-Sensitive Networking

More recent architectures increasingly incorporate Ethernet-based technologies and Time-Sensitive Networking (TSN) mechanisms in order to support higher bandwidth requirements and improved scalability [Ieeb]. TSN extends standard IEEE 802.3 Ethernet with a suite of standards developed by the IEEE 802.1 working group, collectively providing mechanisms for traffic prioritization, time synchronization, bounded latency, and deterministic communication.

Key TSN standards include IEEE 802.1AS for generalized precision time protocol (gPTP)-based clock synchronization [Ieea], IEEE 802.1Qbv for time-aware shaping (TAS), and IEEE 802.1Qbu/802.3br for frame preemption, which allows high-priority frames to interrupt the transmission of lower-priority frames. The Credit-Based Shaper (CBS) defined in IEEE 802.1Qav provides bandwidth reservation for audio-video bridging (AVB) streams. Together, these mechanisms allow TSN networks to support multiple traffic classes with heterogeneous latency and bandwidth requirements on a shared infrastructure.

TSN capabilities are especially relevant for distributed real-time systems where tasks executing on different nodes require predictable communication behavior. In the context of task migration, TSN networks can provide bounded-latency channels for state transfer, coordination messages, and monitoring traffic, enabling more predictable migration latency bounds than best-effort networks. However, the interaction between TSN traffic scheduling, task scheduling on computing nodes, and the timing constraints of distributed applications remains an active area of research [Cra+16].

2.4.2 Service-Oriented Communication and Middleware

In addition to transport-layer networking technologies, the communication middleware used in distributed real-time systems plays a significant role in system integration and runtime behavior. The AUTOSAR Adaptive Platform adopts a service-oriented communication model based on the SOME/IP protocol [Aut], which supports dynamic service discovery and method-call semantics over standard IP networks. The Data Distribution Service (DDS) middleware standard [Omg] provides a publish-subscribe communication model with Quality-of-Service (QoS) policies governing reliability, latency budgets, deadlines, and resource limits, and has seen growing adoption in autonomous and robotic systems through its use as the underlying transport in the Robot Operating System 2 (ROS2) [MKA16].

In distributed task migration and orchestration scenarios, network infrastructure additionally plays an important role in state transfer, coordination, synchronization, and runtime monitoring. Communication latency and bandwidth limitations can significantly influence migration overhead and overall system schedulability. The co-design of communication scheduling policies and task scheduling policies for distributed real-time systems with migration support constitutes one of the open research problems motivating the work presented in this dissertation.

2.5 Summary

This chapter introduced the fundamental concepts relevant to the research presented in this dissertation. Starting from the foundations of real-time systems, scheduling theory, and operating systems support, the discussion progressed to mixed-criticality models

2 Background and Fundamentals

and their associated challenges in resource sharing, mode management, and safety certification. The chapter then addressed distributed real-time system architectures, covering end-to-end timing analysis, the automotive evolution towards centralized and zonal platforms, heterogeneous and edge computing paradigms, and the specific challenges of task migration in real-time contexts. Finally, relevant communication technologies, from legacy fieldbuses to TSN and service-oriented middleware, were reviewed.

The presented concepts collectively establish the theoretical foundation required for understanding the state of the art and the research challenges discussed in the following chapter. In particular, the intersecting requirements of mixed-criticality scheduling, distributed resource management, and deterministic communication under task migration scenarios motivate the design space explored in the remainder of this dissertation.

3 Related Work and State of the Art

The increasing complexity of modern embedded and cyber-physical systems has led to extensive research on architectures that combine deterministic timing guarantees with flexibility and runtime adaptability. Classical real-time systems were designed under static assumptions regarding task sets, execution platforms, and scheduling policies. However, modern automotive and distributed embedded systems increasingly require dynamic adaptation to workload variations, heterogeneous hardware, and runtime reconfiguration. This chapter reviews the most relevant areas of research related to this dissertation, covering real-time scheduling, mixed-criticality systems, distributed embedded architectures, task migration, machine learning for scheduling, and prior work at the chair. Each section concludes by identifying the specific limitations that motivate the contributions of this dissertation.

3.1 Scheduling and Resource Management in Real-Time Systems

Foundational work in real-time scheduling established classical models for periodic task systems and provided schedulability conditions for fixed-priority and dynamic-priority scheduling algorithms [LL73; But11]. Rate Monotonic (RM) and Earliest Deadline First (EDF) scheduling remain the primary reference paradigms, with well-understood optimality properties under their respective assumptions. Response Time Analysis (RTA) [JP86] extended these results to account for blocking and shared resource access under fixed-priority scheduling, providing the analytical foundation that most industrial real-time system design still relies upon.

As systems have grown more complex, the applicability of uniprocessor scheduling theory has become increasingly limited. The transition to multicore and heterogeneous platforms introduces new sources of timing interference that classic models do not capture: shared cache contention, memory bus saturation, NUMA effects, and pipeline stalls caused by speculative execution. These phenomena invalidate simple additive compositional reasoning and have driven a substantial body of research on memory interference analysis and cache-aware scheduling for multicore platforms [Man+13; HGB+24].

3.1.1 Multiprocessor and Distributed Scheduling

Multiprocessor real-time scheduling introduces additional challenges including task migration overhead, synchronization costs, and the fundamental complexity of the multiprocessor feasibility problem. The taxonomy of partitioned, global, and semi-partitioned scheduling strategies [DB11] reflects the trade-off between analytical tractability and runtime efficiency: partitioned approaches allow independent per-core analysis but may fail to schedule task sets that global approaches can handle, while global approaches improve utilization at the cost of migration overhead and substantially harder schedulability analysis.

Megel et al. investigated migration-aware scheduling strategies aimed at minimizing preemption and migration overhead in global multiprocessor scheduling [MSD10]. Their work demonstrates that migration overhead is a first-order concern in practice, not a secondary effect, and must be explicitly modeled in schedulability analysis. However, this work addresses shared-memory multiprocessors and does not extend to distributed heterogeneous systems where migration involves network state transfer.

In distributed systems, scheduling decisions can no longer be considered independently from the communication infrastructure. End-to-end latency analysis for task chains spanning multiple nodes [Dür+19] has emerged as a key research area, particularly in the automotive domain where cause-effect chains connect sensor acquisition, computation, and actuator output across multiple ECUs. The interaction between node-local scheduling and network scheduling in TSN-based distributed systems is an active open problem [Cra+16], with joint optimization of task and network schedules shown to yield significant improvements over independent approaches.

Gap: Existing multiprocessor scheduling research addresses migration cost primarily in shared-memory settings and does not account for criticality differentiation in migration policy decisions. End-to-end timing analysis for distributed systems has advanced significantly but has not been extended to account for the transient scheduling state introduced by runtime task migration.

3.2 Mixed-Criticality Systems

The mixed-criticality model introduced by Vestal [Ves07] and subsequently formalized by Baruah et al. [Bar+11] has generated a large body of research over the past fifteen years. The Burns and Davis survey [BD17], now in its thirteenth edition, documents the breadth of this field and identifies open problems that remain unresolved. The core mechanism of the standard MC model—mode switching from LO to HI upon a WCET overrun, with degradation or suspension of low-criticality tasks—has been extended in multiple directions.

3.2.1 Adaptive and Multicore Mixed-Criticality Scheduling

The Adaptive Mixed Criticality (AMC) scheduler proposed by Burns and Baruah [BB13] introduced per-task mode switching, allowing finer-grained response to individual task overruns rather than a global system-wide mode change. This reduces unnecessary degradation of low-criticality tasks when only a subset of high-criticality tasks approach their pessimistic WCET bounds. Subsequent work explored elastic and imprecise computation models that allow continuous service adjustment rather than binary mode transitions, improving utilization while maintaining safety guarantees.

On multicore platforms, mixed-criticality scheduling must additionally contend with shared resource interference. Cache partitioning and memory bandwidth regulation [Man+13] have been identified as essential mechanisms for providing isolation between criticality levels on shared-cache multicore hardware. The MCTI framework of Hoornaert et al. [HGB+24] provides a layered approach to task-based isolation, combining CPU regulation at the OS level with memory bandwidth control at the hypervisor level, demonstrating that strong isolation is achievable on commercial off-the-shelf (COTS) multicore platforms without specialized hardware.

Flexible and dynamic MC scheduling approaches have also been explored for scenarios where task criticality is not statically fixed [Jia+22]. Jiang et al. propose resource allocation mechanisms that dynamically adjust service levels based on runtime system state, providing a more gradual degradation model than the binary LO/HI switch. Similarly, energy-aware MC scheduling has received growing attention, with surveys documenting the trade-off between power consumption and timing guarantees in battery-constrained embedded systems [Ma+22].

3.2.2 Mixed-Criticality in Distributed and Fault-Tolerant Systems

Extending MC scheduling to distributed systems introduces the problem of mode propagation: when a high-criticality task on one node triggers a mode switch, nodes that share cause-effect chains or communication dependencies with the overrunning task must also adapt their scheduling behavior. This coordination requires either synchronous global mode transitions, which introduce communication overhead and potential blocking, or asynchronous local decisions that may lead to transient inconsistencies. Neither approach has been fully satisfactorily resolved in the literature.

Fault-tolerant mixed-criticality scheduling has been investigated by several groups. The work of Melani et al. [Mel+17] addresses fault-tolerant scheduling of MC task sets on distributed platforms using temporal redundancy, with spatial separation constraints derived from hazard analysis. Reghenzani et al. [RGF22] propose a mixed-criticality approach to fault tolerance in which criticality levels directly govern the degree of redundancy and the recovery time budget allocated to each task. These approaches demonstrate that MC and fault tolerance are complementary rather than conflicting concerns, but they assume static task allocation and do not address the case where the

execution node of a task changes at runtime.

The most directly relevant recent work to this dissertation is that of Baik et al. [BLK22], who propose a task migration and scheduling framework specifically for mixed-criticality systems in vehicle safety applications. Their approach allows low-criticality tasks to temporarily execute on neighboring nodes when their local platform is under high load, effectively extending schedulability by borrowing compute capacity, while preserving high-criticality task guarantees through deadline-based compatibility tests. However, their approach operates at the task activation level rather than providing a full execution state migration mechanism, and does not address the state transfer, checkpoint, and loading overhead involved in migrating a partially-executed task.

Gap: The intersection of MC scheduling with runtime execution state migration, where a task’s in-progress execution, including stack state and accumulated progress, is transferred to a different node, has not been addressed. Existing distributed MC work either assumes static allocation or addresses only coarse-grained failover at the task activation level.

3.3 Distributed Real-Time Systems and Automotive Architectures

Distributed real-time systems consist of multiple computational nodes connected via a communication network, and are the dominant architecture in automotive, aerospace, and industrial automation domains. The automotive domain has undergone particularly rapid architectural evolution in recent years, driven by the growing computational demands of ADAS, autonomous driving, and software-defined vehicle (SDV) functionality.

3.3.1 Automotive Architecture Evolution

The traditional domain-oriented ECU architecture, in which separate ECUs independently manage powertrain, chassis, body electronics, and infotainment, is giving way to centralized and zonal architectures [Ela+24; Ben+23]. In a zonal architecture, ECUs are grouped by physical vehicle zone rather than by function, with high-bandwidth Ethernet backbones connecting zone controllers to one or more central compute platforms. This consolidation reduces wiring harness complexity, a significant factor in vehicle weight and assembly cost, while enabling more powerful centralized processing and simpler over-the-air software updates.

The SDV concept extends this further by treating all vehicle functions as software services deployable and updatable independently of the underlying hardware. Industry adoption is accelerating: major OEMs and Tier-1 suppliers have announced centralized compute platform programs, and the AUTOSAR Adaptive Platform [Aut] provides a standardized software framework targeting these high-performance com-

puting nodes with support for dynamic service deployment and POSIX-compatible execution environments.

From a research perspective, this architectural transition surfaces a critical challenge: as more functions are consolidated onto fewer, more powerful nodes, the consequences of a node failure become more severe, making fault tolerance and runtime adaptation—including task migration—correspondingly more important.

3.3.2 Time-Sensitive Networking

Time-Sensitive Networking (TSN) has emerged as the dominant communication technology for deterministic Ethernet-based distributed embedded systems. The suite of IEEE 802.1 amendments provides mechanisms for time synchronization (802.1AS), time-aware traffic shaping (802.1Qbv), frame preemption (802.1Qbu), and credit-based shaping (802.1Qav), collectively enabling multiple traffic classes with heterogeneous latency and bandwidth requirements to coexist on shared infrastructure.

The scheduling of time-triggered traffic in TSN networks using the Time-Aware Shaper (TAS) has attracted substantial research attention. Stüber et al. [St22] provide a comprehensive survey of TAS scheduling algorithms, documenting the optimization objectives, problem formulations, and evaluation methodologies of existing approaches. A more recent systematic review and experimental study by Xue et al. [XZZ+24] evaluates 17 representative TAS scheduling methods on both simulation and real hardware testbeds, identifying that no single method dominates across all scenarios and that the interaction between network topology, traffic pattern, and scheduling algorithm is highly application-specific.

Joint scheduling of tasks and network traffic in TSN-based distributed real-time systems has been shown to offer significant improvements over independent approaches [Cra+16]. In such systems, the network schedule and the task schedule on each node are mutually dependent: the timing of task activations determines when frames are generated, while the network schedule determines when results become available for downstream tasks. This co-design problem becomes even more complex when tasks may migrate between nodes at runtime.

Gap: Existing TSN scheduling research does not address the case where the task assignment to nodes changes dynamically. Migration introduces transient traffic patterns (e.g. state transfer streams, coordination messages) that are not captured in any existing TSN scheduling model, and may require dynamic reconfiguration of the network schedule during a migration event.

3.4 Task Offloading and Orchestration in IoT and Edge Systems

In parallel to research on distributed real-time systems, a large body of work in the Internet of Things (IoT), edge computing, and fog computing has investigated the

problem of task offloading and resource orchestration across heterogeneous computing nodes. These works provide relevant conceptual models and optimization techniques, even though their primary target is best-effort or soft real-time workloads.

Mao et al. [Mao+17] studied computation offloading strategies in mobile edge computing (MEC) systems, demonstrating that optimal offloading decisions must jointly account for communication cost, computation latency, and energy consumption, and that these factors interact in non-trivial ways depending on dynamic system conditions. You et al. [You+17] extended this to joint communication and computation resource allocation, showing that network and compute constraints must be optimized simultaneously rather than independently.

Container-based virtualization has been increasingly adopted for edge computing deployments, offering process isolation and portability with lower overhead than full virtual machines. Empirical studies of checkpoint/restore in containerized environments [Tos23] demonstrate that container migration latency is dominated by the size of the process memory image and network transfer time rather than by container initialization overhead, a finding that directly informs the Virtual ECU migration strategy investigated in this dissertation. Morabito et al. [Mor17] evaluated the performance overhead of container virtualization on IoT edge devices, finding that containerization imposes measurable but manageable overhead for compute-bound workloads while adding significant latency for storage and network operations.

A fundamental limitation of IoT and edge offloading research is its assumption of relaxed or best-effort timing constraints. Offloading frameworks prioritize throughput, energy efficiency, or cost, and treat latency as a soft optimization objective rather than a hard real-time requirement. This limits the direct applicability of these approaches to safety-critical embedded systems. Nevertheless, the architectural patterns for dynamic task placement, hierarchical resource management, and state transfer developed in this body of work provide valuable reference models for the Virtual ECU migration tier explored in this dissertation, where non-safety-critical tasks may be offloaded to edge resources under controlled conditions.

Gap: Edge and IoT offloading frameworks do not provide hard real-time guarantees and do not integrate with onboard mixed-criticality scheduling. The conditions under which edge offloading is safe for tasks at the lowest criticality levels of an automotive MC system—and the latency and reliability constraints that the communication path must satisfy—have not been systematically studied.

3.5 Task Migration in Real-Time Systems

Task migration has been studied from several angles in the real-time systems literature: as a mechanism for load balancing in multiprocessor scheduling, as a fault recovery mechanism in distributed systems, and more recently as a component of adaptive orchestration frameworks.

3.5.1 Migration Overhead and Schedulability

In multiprocessor real-time scheduling, migration overhead is a well-recognized first-order concern. Megel et al. [MSD10] demonstrated that migration and preemption costs must be explicitly bounded and included in schedulability analysis, and proposed scheduling strategies that minimize the number of migrations while preserving global feasibility. Their work established that the commonly used simplification of treating migration as instantaneous is untenable in practice.

More recent work on migration-aware scheduling in heterogeneous multiprocessor systems [DB11] has confirmed that the overhead of migration—including cache repopulation, TLB invalidation, and pipeline refilling on the destination processor—can dominate the actual task execution time for short-period tasks, making migration frequency a critical design parameter.

3.5.2 Fault-Tolerant Migration in Embedded Systems

In fault-tolerant embedded systems, migration is primarily used as a recovery mechanism triggered by hardware failure. Chen and Lyu [CL12] proposed a FlexRay-based framework in which backup ECUs take over execution when the primary fails. However, their approach operates at coarse granularity: entire ECU function groups are re-assigned, rather than providing fine-grained task-level migration with state preservation. The checkpoint-restore approach to fault-tolerant migration, surveyed comprehensively by Elnozahy et al. [Eln+02], provides a principled mechanism for capturing and restoring execution state, but the original work targets message-passing cluster systems rather than resource-constrained embedded nodes with hard real-time constraints.

Checkpoint-based migration in real-time embedded contexts introduces the specific challenge that the checkpointing operation itself consumes time and memory resources, and this overhead must be budgeted in the schedulability analysis of the checkpointed task. If checkpointing is performed periodically, the checkpoint interval must be chosen to balance recovery cost (coarser intervals mean more re-execution after recovery) against runtime overhead (finer intervals impose more frequent interruptions).

3.5.3 Runtime Migration in Distributed Real-Time Systems

Research on runtime task migration specifically for distributed real-time systems remains limited. The DFT4FTT project investigated adaptive task allocation in distributed embedded systems supporting real-time, reliability, and adaptivity requirements, proposing greedy online heuristics for migrating safety-critical tasks in response to permanent faults while adjusting soft real-time task service parameters on the target node [FPY13]. This work represents one of the closest precedents to the approach developed in this dissertation, but does not address mixed-criticality scheduling or the ELF-based execution state transfer mechanisms required for heterogeneous embedded targets.

The interaction between dynamic task migration and TSN network scheduling has been highlighted as an open problem by recent work on joint task and network scheduling [Cra+16; XZZ+24]: a migration event changes the traffic pattern on the network, potentially invalidating the network schedule that was computed for the static task assignment.

Gap: No existing work provides a complete runtime task migration system for distributed embedded real-time systems that (a) differentiates migration policy and mechanism by criticality level, (b) handles the full execution state transfer including checkpoint and ELF-based loading on heterogeneous embedded targets, and (c) accounts for the schedulability implications of migration events on both source and destination nodes under a mixed-criticality scheduling framework.

3.6 Machine Learning for Scheduling and Deployment

Machine learning techniques have been increasingly applied to scheduling and resource allocation problems due to their ability to approximate complex optimization objectives that are intractable for exact methods. In the real-time scheduling domain, the primary motivation is the computational cost of exact schedulability analysis for large task sets: algorithms such as response time analysis scale poorly with the number of tasks and become infeasible for runtime use.

Early work explored neural network approaches for offline schedulability prediction and task ordering [CM94; DW04]. These approaches demonstrated that learned predictors could achieve high accuracy on synthetic task sets, but were limited to offline design-time use and did not generalize well across task set distributions.

More recent research at the chair, conducted across multiple theses [Tai19; Utz19; Bli19], applied neural network-based feasibility prediction to the migration planning problem. The key insight is that predicting schedulability, a binary decision, is faster than computing exact response times, making runtime replanning tractable even for large task sets. The trade-off is that neural network predictions are not unconditionally safe: false positive predictions (declaring a task set feasible when it is not) can occur, and the rate of such errors must be characterized and bounded for safety-critical use.

Reinforcement learning (RL) has been applied to dynamic scheduling and routing problems in TSN and edge environments, with promising results in simulation but limited deployment in safety-critical real-time systems. The fundamental challenge is that RL policies do not provide formal timing guarantees: a learned policy may perform well on average but cannot be certified to satisfy hard deadlines under all possible inputs. Bridging the gap between the performance of ML-based approaches and the certification requirements of safety-critical domains remains an open research problem.

Gap: Existing ML-based scheduling and planning tools at the chair were developed for single-criticality task sets and do not incorporate mixed-criticality parameters.

Extending one of these tools to support MC task sets, where each task has multiple WCET estimates and the feasibility prediction must account for potential mode switches, is a secondary contribution of this dissertation.

3.7 Prior Work at the Chair

Research at the Chair of Operating Systems at TUM has produced a series of projects and theses directly relevant to this dissertation. These works form the technical and conceptual foundation for task migration and adaptive execution in distributed embedded systems. However, their focus differs fundamentally from the approach presented in this dissertation: prior work primarily investigates migration as a planning, orchestration, or checkpointing problem, while assuming that a runtime execution substrate for stateful task migration is either available or abstracted away.

The KIA4SM project [EKB15] introduced a cooperative integration architecture for smart mobility systems based on Genode OS and Fiasco.OC. It demonstrated the feasibility of distributed automotive software components and introduced a two-stage migration concept (planning and execution). However, the execution stage relies on system-level abstractions and does not provide a lightweight mechanism for directly transferring and resuming task execution state on heterogeneous embedded targets.

The HaCRoM project extended checkpoint-restore mechanisms for fault-tolerant task migration in real-time systems. While this work established important empirical insights into checkpoint overhead and feasibility, it primarily focuses on state capture and recovery as a fault tolerance mechanism. The actual execution of migrated tasks is still assumed to be supported by the underlying runtime system, rather than being implemented as a minimal operating-system-level capability.

The MaLSAMi project shifted focus toward migration decision-making and planning using machine learning-based schedulability prediction. A series of theses by Taieb [Tai19], Utz [Utz19], and Blieninger [Bli19] improved the accuracy of feasibility prediction models for task allocation and migration decisions. However, these approaches operate at the level of decision support and do not address how a task is actually executed after migration, particularly in heterogeneous embedded environments.

A parallel line of work by Delgadillo [Del21] transitioned parts of the execution environment to FreeRTOS, enabling experiments closer to embedded hardware constraints. Nevertheless, the system still assumes that task execution is locally bound and does not provide a mechanism for transferring partially executed runtime state between nodes with different execution contexts.

In summary, prior work at the chair has progressively addressed migration planning, checkpointing, and feasibility prediction, but none of these systems implement migration as a first-class execution primitive within a lightweight embedded operating system. Instead, migration is treated as a higher-level orchestration decision, while the

execution of migrated tasks remains outside the scope of the runtime system itself.

None of the prior projects at the chair integrate mixed-criticality concepts into a complete runtime migration execution system. More importantly, none provide a lightweight OS-level mechanism for stateful task migration across heterogeneous embedded nodes.

The contributions of this dissertation address this gap by introducing Axcan OS, a FreeRTOS-based execution substrate that enables stateful task migration as a first-class primitive, including execution state capture, transfer, and resumption on heterogeneous embedded platforms. This runtime capability forms the foundation on which higher-level scheduling, mixed-criticality management, and planning mechanisms operate.

3.8 Research Gap Summary

The reviewed literature demonstrates substantial progress across real-time scheduling theory, mixed-criticality systems, distributed embedded architectures, communication networks, and task migration research. Classical real-time scheduling provides a strong analytical basis for fixed and dynamic priority systems under static task assumptions. Mixed-criticality research extends these models with adaptive execution modes and controlled degradation strategies. Distributed real-time systems and Time-Sensitive Networking (TSN) provide deterministic communication foundations for networked embedded systems. Finally, task migration and checkpointing techniques explore mechanisms for relocating computation, primarily in the context of fault tolerance or load balancing.

Despite these advances, existing approaches share a fundamental architectural limitation: *task execution is assumed to be bound to a fixed runtime execution context, and migration is treated as either a high-level scheduling decision or a coarse-grained fault recovery action, rather than as a continuous, OS-supported execution capability.* In all existing models, the operating system is assumed to execute tasks locally, while migration is abstracted as task reactivation, restart, or external orchestration.

This abstraction leads to a missing execution layer in current distributed embedded systems: there is no lightweight operating system substrate that enables **stateful, partially executed task migration across heterogeneous nodes as a first-class runtime primitive**. As a result, several fundamental limitations remain unresolved:

1. **Absence of OS-level execution state migration mechanisms.** Existing checkpoint-restore and migration approaches operate as external mechanisms or framework-level services. They do not integrate execution state capture, transfer, and resumption directly into a lightweight real-time operating system. Consequently, partially executed tasks cannot be transparently migrated and resumed across heterogeneous embedded nodes with minimal runtime overhead.
2. **Static execution binding in distributed real-time systems.** Current distributed

scheduling models assume a fixed mapping between tasks and execution nodes. Migration, when considered, is modeled as a discrete reassignment event rather than a runtime continuation of execution state. This prevents accurate modeling of transient execution phases, where a task is simultaneously influenced by source-side execution state, transfer latency, and destination-side scheduling constraints.

3. **Incompatibility of migration with deterministic communication schedules.** TSN-based systems assume statically known traffic patterns derived from fixed task-to-node assignments. Stateful task migration violates this assumption by introducing dynamic, time-varying communication flows, including execution state transfer and coordination traffic, which are not represented in existing offline-computed network schedules.
4. **Lack of real-time capable offloading with execution continuity.** Edge and IoT offloading frameworks support dynamic task placement across heterogeneous resources, but treat tasks as restartable computation units. They do not provide mechanisms for preserving execution continuity under hard real-time constraints, nor do they integrate with embedded operating system scheduling semantics.
5. **Separation of learning-based decision models from execution mechanisms.** Machine learning approaches for scheduling and migration focus on predicting feasibility or optimizing allocation decisions, but assume that execution after migration is handled by an external runtime system. They do not model or control the execution state transfer process, nor its impact on real-time schedulability in mixed-criticality systems.

These limitations converge to a single fundamental gap: **the absence of a lightweight operating system execution substrate that enables stateful task migration as a first-class primitive in distributed heterogeneous real-time systems, while preserving real-time constraints during execution state transfer and resumption.**

This dissertation addresses this gap through the design and implementation of Axcan OS, a FreeRTOS-based execution substrate that integrates execution state capture, transfer, and resumption directly into the runtime system. On top of this substrate, higher-level mechanisms for scheduling, mixed-criticality management, and migration planning can operate without assuming static task-to-node bindings, enabling a new class of adaptive distributed real-time systems.

4 Approach

This chapter develops the design of a breathable distributed real-time system. It begins with the system vision and the conditions that trigger reconfiguration (Sections 4.1 and 4.2), formalises the workload, the platform and the notion of a migration event (Section 4.3), states the assumptions under which the design is developed (Section ??) and the requirements it must satisfy (Section ??), and derives from these the architecture (Section 4.4) and the criticality-stratified migration design (Section 4.5) that the following chapters realise. The design decisions taken along the way, and the alternatives rejected, are collected in Section 4.6. The chapter closes by delimiting the safety claims made in this work (Section 4.7) and by relating each mechanism to the chapter in which it is described (Section 4.8).

Parts of the architecture and of the system model presented here were published in [delgadillo_rage26].

4.1 Breathability

Conventional distributed real-time systems fix the assignment of tasks to computing nodes at design time. The assignment is then the subject of an offline schedulability analysis, and the guarantees derived from that analysis hold for as long as the assumptions underlying it hold: the task set does not change, and the set of functioning nodes does not change. Neither condition is satisfied in a modern vehicle, where computing resources appear and disappear as devices fail, restart for updates, are powered down for energy reasons, or as edge infrastructure enters and leaves communication range.

This dissertation therefore develops a system that is *breathable*. A distributed real-time system is called breathable if the assignment of tasks to nodes is revisable at runtime, and if every revision preserves the timing guarantees of tasks in order of their criticality: guarantees given to tasks of higher criticality are never relaxed in favour of tasks of lower criticality. The metaphor is deliberate — the system expands onto additional resources when they become available and contracts onto the remaining ones when they do not, shedding load from the least critical end of the task set rather than failing as a whole.

Breathability is triggered by two categories of condition:

1. *Changes to the task set.* Tasks are added to or removed from the global task queue, or the load they impose changes. An increase may exceed the capacity of the

current assignment; a decrease may permit consolidation onto fewer nodes and the powering down of the remainder.

2. *Changes to the resource set.* Physical devices are added or removed, whether through hardware failure, restart after an update, or deliberate power management. Virtual devices hosted on Mobile Edge Computing (MEC) infrastructure become reachable or unreachable as a function of vehicle position, and the latency of the link to them varies.

Reacting to either category requires the same underlying capability: moving a running task from the node currently executing it to another node, without losing the progress it has made and without violating the timing guarantees of that task or of any other. Providing that capability, and establishing the conditions under which it preserves timing guarantees, is the subject of this dissertation. The decision of *which* tasks to move and *where* is a separate problem, addressed in prior work of this research group [delgadillo_jlpea22] and treated here as an external component (Assumption A6).

4.2 Use Cases

The following four scenarios make the triggering conditions concrete. They are referred to throughout the dissertation and form the basis of the evaluation scenarios in Chapter ??.

UC1 — Physical device failure. A physical ECU becomes unavailable. Tasks of high criticality that were executing on it must resume elsewhere with minimal and, above all, bounded downtime; the design provides for this by maintaining a prepared backup node for such tasks. Tasks of low criticality, both those displaced from the failed device and those already resident on the node that absorbs the load, are redistributed across the remaining hardware. Where the remaining hardware cannot accommodate the entire task set, tasks of high criticality are served first and tasks of low criticality skip jobs, or are deferred entirely, until load decreases or capacity is restored. This use case exercises Requirements R1, R3 and R4.

UC2 — Change in task load. The load imposed by the task set rises or falls. A rise may require redistribution across nodes, or the activation of a node previously held in standby, in order to reach a configuration in which every task is schedulable. A fall permits the converse: consolidation onto fewer nodes and the powering down of the remainder for energy efficiency. In both directions, the migration of highly critical tasks is avoided except where no alternative exists, since each migration introduces an additional opportunity for failure; this preference is expressed as a penalty in the

planning component (Assumption A6) rather than as a prohibition. This use case exercises Requirements R3 and R6.

UC3 — Capability-driven offload to the MEC. MEC infrastructure becomes available and hosts resources a task can exploit — greater computational capacity, or sensor data from beyond the vehicle. A task able to benefit is migrated to a virtual node. Because the availability of the infrastructure is a function of vehicle position, such a task retains a counterpart on physical hardware that provides basic functionality, while the migrated instance provides enhanced functionality for as long as the infrastructure remains reachable. A path-planning function illustrates the pattern: the local counterpart follows the current route safely, while the offloaded instance computes alternative routes using traffic and incident data available only at the edge. This use case exercises Requirements R4 and R5.

UC4 — Load-driven offload to the MEC. MEC infrastructure is available and the load on the physical devices is critically high, for instance following the device failure of UC1. Tasks of low criticality that would otherwise be starved are offloaded to virtual nodes for as long as the infrastructure remains reachable, accepting the additional latency of the external link in exchange for continued service. An infotainment streaming service is a representative example. The results such tasks produce must be forwarded back into the vehicle network, which requires a task-independent forwarding mechanism on the vehicle side. This use case exercises Requirements R1 and R3.

UC1 and UC2 are driven by changes within the vehicle and are served by migration between physical nodes. UC3 and UC4 both involve virtual nodes but differ in what motivates the migration — capability in UC3, capacity in UC4 — and therefore in what happens when the link is lost: in UC3 the local counterpart continues unaffected, whereas in UC4 the offloaded task must be recalled or dropped. This distinction is developed in Section 4.5.

4.3 System Model

This section introduces the notation used throughout the remainder of this dissertation. The model is deliberately compact: it captures only those properties of tasks, nodes and migration events that are needed to state the design requirements of Section ??, to formulate the migration cost model of Chapter 6, and to interpret the measurements of Chapter ?. A summary of the symbols is given in Table 4.1.

4.3.1 Nodes and Sub-Nodes

The system follows a master–worker model. A single *master node* coordinates a set of *worker nodes* that execute the task set. Where the distinction is not at issue, worker

4 Approach

nodes are referred to simply as nodes.

A *node* is a device: a physical Electronic Control Unit (ECU) within the vehicle, or a virtual ECU hosted on Mobile Edge Computing (MEC) infrastructure outside it. A node comprises one or more *sub-nodes*, a sub-node being an independently scheduled instance of the execution platform, owning a scheduler and a task set of its own. A single-core node has one sub-node. A multi-core node operated in Asymmetric Multiprocessing (AMP) mode has one sub-node per core, of which one is designated the *master core* and acts as the node's single point of contact towards the master node; the remainder are *slave cores*.

The distinction matters because the properties relevant to the design divide between the two levels. Communication properties are properties of a device: every core of an ECU reaches the master node over the same link and shares the same hardware clock. Scheduling properties are properties of a core: each runs its own scheduler and, under the reserved-core realisation of Section 5.3.3, may carry a criticality ceiling different from that of its neighbours. Tasks are assigned to sub-nodes, not to nodes.

The set of nodes is denoted $\mathcal{N} = \{v_1, \dots, v_m\}$, with

$$v_j = (\theta_j, \Sigma_j, \beta_j, \delta_j, \Delta_j, \Pi_j), \quad (4.1)$$

where $\theta_j \in \{\text{physical}, \text{virtual}\}$ is the node type, Σ_j the set of system services the node can provide (network access, specific sensors, peripherals), β_j the bandwidth of the link to the master node, δ_j the bounded latency of that link, Δ_j the bound on the deviation of the node's clock from the global reference, and Π_j its set of sub-nodes.

Each sub-node $\pi_k \in \Pi_j$ is characterised by

$$\pi_k = (f_k, \mu_k, L_k^{\max}), \quad (4.2)$$

with f_k the processor frequency, μ_k the memory available to guest tasks, and $L_k^{\max}$ the *criticality ceiling* introduced in Section 4.3.3. Where a property of the node containing a sub-node is required, it is written with the node's index; thus δ_j is the link latency applicable to every $\pi_k \in \Pi_j$.

Nodes of type *virtual* are hosted on MEC infrastructure. Their defining property in this model is not their virtualisation technology but the fact that δ_j is neither controlled by the vehicle nor bounded by an in-vehicle network, which is the formal reason for the criticality ceiling carried by their sub-nodes.

All nodes share a common instruction set architecture, ARM AArch64 in the realisation described in Chapter 5. Heterogeneity is therefore supported at the level of the software stack, the hardware peripherals and the processor frequency, but not at the level of the instruction set (Assumption A5).

4.3.2 Tasks, Jobs and Criticality

The workload is a set of independent tasks $\Gamma = \{\tau_1, \dots, \tau_p\}$ held in a global task queue by the master node. Each task is characterised by the tuple

$$\tau_i = (\chi_i, \mathbf{C}_i, T_i, D_i, N_i, r_i, S_i, E_i, \Sigma_i), \quad (4.3)$$

with the following components:

- $\chi_i \in \mathcal{L}$ — the *criticality level* of the task, drawn from the ordered set $\mathcal{L}$ defined in Section 4.3.3.
- $\mathbf{C}_i = (C_i(\ell))_{\ell \in \mathcal{L}}$ — the vector of worst-case execution time (WCET) estimates, one per criticality level, following the mixed-criticality task model of Vestal [Ves07]. The estimates are monotonically non-decreasing in the criticality level, $\ell_1 \leq \ell_2 \Rightarrow C_i(\ell_1) \leq C_i(\ell_2)$, reflecting the increasing pessimism required by higher assurance levels. Execution times are platform-dependent and are specified relative to a reference processor frequency; the estimate applicable on sub-node π_k is written $C_i^k(\ell)$.
- T_i — the period, and $D_i \leq T_i$ the relative deadline (constrained-deadline model).
- $N_i \in \mathbb{N} \cup \{\infty\}$ — the number of jobs the task is to execute, and r_i its release time. This parameter unifies the treatment of periodic and sporadic workloads: a task with $N_i = \infty$ is a conventional periodic task, a task with $N_i = 1$ models a single sporadic activation, and finite $N_i > 1$ models a bounded batch of activations, a form used for lower-criticality workloads that may be admitted or deferred in batches (Assumption A2).
- S_i — the size of the task's *checkpoint*, i.e. the serialised state that must be preserved across a migration event, and E_i the size of its executable image. These two parameters do not appear in classical real-time task models. They are introduced here because, in a system where task-to-node binding is revised at runtime, the cost of that revision is dominated by the transfer of S_i and E_i ; they are consequently the inputs to the migration cost model of Section ??.
- Σ_i — the set of system services the task requires. A task may only be assigned to a sub-node of a node v_j with $\Sigma_i \subseteq \Sigma_j$.

A *job* $J_{i,q}$ is the q -th activation of task τ_i . Jobs follow the sense–think–act pattern common in cyber-physical systems: a job reads its inputs, restores the task state produced by $J_{i,q-1}$, computes, emits its outputs, and persists the state for $J_{i,q+1}$. The task state is the only information that crosses the boundary between consecutive jobs; this property is what makes the job boundary a natural quiescent point and is the basis for Assumption A3.

4.3.3 Criticality Levels and Node Ceilings

Criticality levels are drawn from a finite ordered set $\mathcal{L}$ modelled on the Automotive Safety Integrity Levels (ASIL) of ISO 26262 [Iso]. Conceptually, $\mathcal{L}$ may contain any number of levels; the design of the migration strategies in Chapter 6 is formulated for arbitrary $|\mathcal{L}|$, while the implementation and the evaluation use the two-level instantiation $\mathcal{L} = \{\text{LO}, \text{HI}\}$ that is standard in the mixed-criticality literature [BD17]. The rationale for this restriction is that the design problems addressed here occur at the two ends of the criticality spectrum — bounded, resource-expensive migration at the top, and flexible, resource-efficient migration at the bottom — and that intermediate levels are addressed by combining the two mechanisms rather than by introducing a third one; this is discussed further in Section ??.

Each sub-node carries a criticality ceiling $L_k^{\max}$: the highest criticality level a task may have to be admissible on that sub-node. Ceilings are the mechanism by which platform properties are translated into scheduling constraints. In particular, virtual nodes are assigned a ceiling strictly below the highest level, because the latency of the link to MEC infrastructure is not bounded by any in-vehicle mechanism and because the availability of the infrastructure itself is a function of vehicle position rather than of system state. A task τ_i is admissible on sub-node π_k of node v_j only if

$$\chi_i \leq L_k^{\max} \quad \wedge \quad \Sigma_i \subseteq \Sigma_j \quad \wedge \quad \text{the local schedulability test of } \pi_k \text{ admits } \tau_i. \quad (4.4)$$

4.3.4 Migration Events

A *migration event* is denoted

$$\mathcal{M}(\tau_i, \pi_{\text{src}}, \pi_{\text{dst}}, t), \quad (4.5)$$

the transfer of the execution of task τ_i from source sub-node π_{src} to destination sub-node π_{dst} , initiated at time t . A migration event is *admissible* if Equation 4.4 holds for τ_i on π_{dst} , and it is *timing-preserving* if no job of τ_i , and no job of any task resident on either sub-node, misses its deadline as a consequence of the event. Establishing the conditions under which migration events are timing-preserving is the object of research question RQ1 and is treated quantitatively in Section ??.

Migration events are triggered by the master node in response to two categories of condition, as introduced in Section 4.1: a change in the task set Γ , or a change in the node set Π or in the properties of its members. The decision procedure that selects which migrations to perform is outside the scope of this dissertation and is treated as an external component (Assumption A6).

4.3.5 Time

Nodes maintain a common notion of time through the Precision Time Protocol (PTP). The clock of node v_j deviates from the global reference by an offset bounded by Δ_j ,

Table 4.1: Summary of notation.

Symbol	Meaning
$\mathcal{N}, v_j$	set of nodes (devices); individual node
Π_j, π_k	sub-nodes of node v_j ; individual sub-node
θ_j, Σ_j	node type; services offered by the node
β_j, δ_j	link bandwidth and bounded latency of node v_j
f_k, μ_k	processor frequency and guest memory of sub-node π_k
$L_k^{\max}$	criticality ceiling of sub-node π_k
Σ_i	services required by task τ_i
Γ, τ_i	task set; individual task
$\chi_i, \mathcal{L}$	criticality of a task; ordered set of criticality levels
$C_i(\ell), C_i^k(\ell)$	WCET at level ℓ ; WCET at level ℓ on sub-node π_k
T_i, D_i, D'_i	period, global relative deadline, compensated deadline
$N_i, r_i, J_{i,q}$	job count, release time, q -th job of τ_i
S_i, E_i	checkpoint size, executable image size
$\Delta_j, \Delta^{\max}$	clock offset bound of node v_j ; system-wide bound
$\mathcal{M}(\cdot)$	migration event
Δ_{mig}	total migration cost (Section ??)

and $\Delta^{\max} = \max_j \Delta_j$ denotes the system-wide bound. Because the sub-nodes of a node share its hardware clock, the offset is a property of the node and applies uniformly to all of its sub-nodes. Deadlines are specified globally and compensated locally: a task with global relative deadline D_i is scheduled on a sub-node of v_j against the adjusted deadline

$$D'_i = D_i - \Delta_j - \delta_j. \quad (4.6)$$

Because both correction terms are properties of the node rather than of the task, the adjustment is uniform across the task set of every sub-node of that node and therefore preserves the relative ordering of deadlines under EDF-based policies. It does, however, reduce the available slack on every task resident on the node, and thus has a schedulability cost; this cost is quantified in Section ??.

The model of Section 4.3 rests on a number of assumptions. They are stated explicitly here, together with the reason each is made and an indication of whether it is a fundamental restriction of the approach or a scoping decision that could be relaxed without invalidating the design.

A1 Task independence. Tasks share no data and impose no precedence constraints on one another; consequently no mutual-exclusion protocol is required between them. *Rationale:* resource-sharing protocols for mixed-criticality systems are an active research area in their own right [BD17], and admitting shared resources would conflate the cost of migration with the cost of releasing and re-acquiring

4 Approach

locks across nodes. Isolating the migration mechanism requires isolating the tasks. *Status*: scoping decision. It is the strongest assumption made here and the most consequential for automotive realism, since sensor-fusion and perception pipelines are typically chains of dependent components. Relaxing it requires a distributed resource-sharing protocol and is identified as future work in Section ??.

A2 Periodic task model with bounded job count. All tasks are treated as periodic with a job count N_i , as defined in Equation 4.3. *Rationale*: it unifies periodic and sporadic workloads under a single scheduling and migration treatment without weakening the analysis, since a sporadic task with minimum inter-arrival time T_i is schedulable whenever the corresponding periodic task is. *Status*: scoping decision with no significant loss of generality.

A3 Migration at job boundaries. A migration event transfers a task only between two of its jobs; a job in progress is never relocated mid-execution. *Rationale*: the job boundary is the point at which the task's complete state is, by construction, contained in the user-defined checkpoint structure (Section 4.3.2). Relocating a job in progress would require capturing the program counter, stack and heap, which imposes both a substantially larger state transfer and kernel-level snapshot machinery incompatible with the memory footprint targeted here. *Status*: fundamental to the approach. It has the direct consequence that the observable disruption of a migration is bounded by the task period rather than by the transfer time, and it is therefore the assumption that makes the timing analysis of Chapter 6 tractable. Its granularity limitation is partially mitigated by intra-job checkpoints (Section 5.6.3).

A4 Completeness of the declared state. The state structure declared by the application developer captures all information required to resume the task correctly on another node. *Rationale*: the checkpoint mechanism is cooperative rather than transparent; correctness of migration is therefore a property of the guest task, not of the platform alone. *Status*: fundamental, and a genuine burden placed on the application developer. Chapter 5 describes the library support provided to reduce this burden; verification of state completeness is not addressed.

A5 Common instruction set architecture. All nodes share the ARM AArch64 instruction set. *Rationale*: a single position-independent binary per task can then be deployed to any node. The alternatives — maintaining one compiled binary per target, or introducing an intermediate representation with just-in-time compilation — either multiply the artefacts the master node must manage and transfer, or add a runtime engine whose execution time is difficult to bound. *Status*: fundamental to the cross-platform loading mechanism. Heterogeneity in software stack, peripherals and clock frequency is supported; heterogeneity in instruction set is not.

- A6 Migration planning as an external component.** The procedure that decides which tasks should execute on which nodes is treated as a black box that emits admissible assignments. *Rationale:* task-to-node partitioning has been addressed in prior work of this research group [delgadillo_jlpea22; Bli19], and the contribution of this dissertation lies in the execution of the resulting migrations rather than in the decision itself. The interface to this component, and the mixed-criticality extensions required of it, are described in Chapter ?? *Status:* scoping decision.
- A7 Reliable master node.** The master node is assumed to be available and correct; it is not replicated. *Rationale:* the centralised model removes the need for distributed consensus and permits the partitioning decision to run on a node with substantially greater computational resources than the ECUs. *Status:* scoping decision, and an acknowledged single point of failure. A production deployment requires master node redundancy or distributed coordination; this is discussed in Section ??.
- A8 Bounded in-vehicle communication latency.** Communication between the master node and physical nodes exhibits a latency bounded by δ_j . No such bound is assumed for links to virtual nodes. *Rationale:* the in-vehicle network in the evaluation setup is a switched Ethernet segment without deterministic guarantees, and δ_j is therefore an empirical rather than an analytical bound. Obtaining an analytical bound requires Time-Sensitive Networking (TSN), which the architecture accommodates but which is not part of the present realisation. *Status:* scoping decision; it is the reason the timing results of Chapter ?? are stated relative to a measured δ_j . The absence of the corresponding assumption for virtual nodes is deliberate and is what confines them to a criticality ceiling below the highest level.
- A9 Trusted deployment channel.** Executable images and checkpoints received by a node are assumed authentic; no authentication, encryption or integrity verification beyond a transmission checksum is performed. *Rationale:* the mechanisms required — signed binaries, attested loading, encrypted transport — are well understood in isolation but interact with the timing analysis in ways that would obscure the migration costs this work sets out to characterise. *Status:* scoping decision. It is noted explicitly because dynamic loading of executable code received over a vehicle-to-infrastructure link is a significant attack surface, and any deployment of this architecture must address it.
- A10 Certification is out of scope.** The platform is designed to be compatible with the requirements that ISO 26262 imposes on the temporal behaviour of safety-related functions, but no certification argument is constructed and no component of the system is certified. *Rationale and status:* discussed in detail in Section 4.7, which states precisely which properties are claimed and which are not.

4 Approach

The use cases of Section 4.2, interpreted under the model and assumptions above, yield the following requirements on the system. They are referenced throughout the remainder of the dissertation.

- R1 Execution transfer.** It shall be possible to halt the execution of a task on one node and resume it on another, such that the task continues to produce results without manual intervention and without duplicate or omitted jobs.
- R2 State preservation.** A migration event shall preserve the task state required for subsequent jobs to produce correct results. Progress accumulated over previous jobs shall not be lost.
- R3 Criticality differentiation.** Both the decision to migrate and the mechanism by which migration is executed shall depend on the criticality of the task. Higher-criticality tasks shall be favoured whenever resources are contended, and shall never be delayed by the migration of lower-criticality tasks.
- R4 Timing preservation.** The timing behaviour of the system during and after a migration event shall remain analysable. For tasks at the highest criticality level the migration cost shall be bounded and the bound shall be established analytically; for lower levels a probabilistic characterisation is sufficient.
- R5 Platform portability.** A task shall be deployable, without recompilation, to any node satisfying Equation 4.4, irrespective of that node's software stack or virtualisation status.
- R6 Resource proportionality.** The runtime and memory overhead imposed by the migration infrastructure shall be compatible with ECU-class hardware, and the resources reserved for a given migration strategy shall be proportionate to the criticality of the tasks it serves.

Requirements R1 and R2 are common to any task migration mechanism. Requirements R3 and R4 are what distinguish this work from the prior art surveyed in Chapter 3, and jointly they imply that a single migration mechanism is insufficient: a mechanism fast and predictable enough for R4 at the highest criticality level necessarily reserves resources on nodes that are not executing the task, which violates R6 if applied to the whole task set. The criticality-stratified design of Section 4.5, with distinct strategies per criticality tier, follows directly from this tension.

4.4 Architectural Overview

The model of Section 4.3 is realised by the architecture shown in Figure 4.1. A reconfiguring situation triggers a redistribution of the tasks held in the global task queue. The queue and the current node set Π are supplied to the partitioning component,

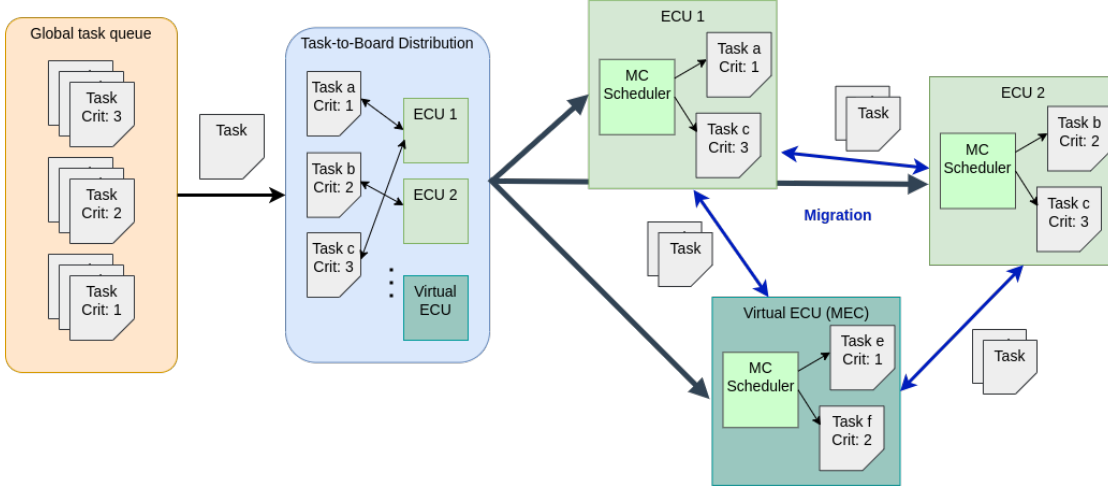


Figure 4.1: General system overview: the global task queue is partitioned onto physical and virtual nodes, each running a mixed-criticality scheduler, with migration events transferring tasks between them.

which computes a task-to-node mapping subject to Equation 4.4. Where the mapping assigns a task to a node other than the one currently executing it, a migration event $\mathcal{M}(\tau_i, \pi_{\text{src}}, \pi_{\text{dst}}, t)$ is issued and executed by the mechanisms of Chapter 6.

The architecture comprises four kinds of component.

The master node maintains the global task queue, tracks the set of available nodes and their reported health, triggers the partitioning component, and stores the artefacts required to start a task on any node: the executable image, the most recent checkpoint, and the task’s parameters according to Equation 4.3. It is the single point of contact for every node and, under Assumption A7, is assumed reliable.

The *partitioning component* computes task-to-node mappings. It is treated as a black box (Assumption A6); the interface it must satisfy, and the extensions required of it to account for criticality and for migration cost, are described in Chapter ??.

Physical nodes execute the majority of the task set. Each runs the operating system described in Chapter 5, which provides a mixed-criticality scheduler, dynamic task loading, and the checkpoint mechanism. A physical ECU with several cores contributes one sub-node per core under AMP, with the master core mediating between the master node and the remaining cores. Nodes may be held in standby or powered down entirely to conserve energy, and returned to service when UC1 or UC2 requires them.

Virtual nodes are containerised instances of the same operating system hosted on MEC infrastructure. They extend the capacity of the system and provide access to services unavailable in the vehicle, but their sub-nodes carry a criticality ceiling below the highest level for the reasons given in Section 4.3.3.

Two properties of this arrangement are worth stating explicitly. First, the same task binary executes on physical and virtual nodes alike (Assumption A5), so the master

node stores one artefact per task rather than one per task and platform. Second, because tasks are assigned to sub-nodes rather than to nodes, migration between cores of one ECU and migration between ECUs are the same operation at the level of the model, differing only in the cost of the transfer — a property the strategies of Section 4.5 exploit.

4.5 Criticality-Stratified Migration

Requirements R3, R4 and R6 cannot be satisfied by a single migration mechanism. A mechanism that bounds migration cost tightly enough to preserve the guarantees of the most critical tasks must avoid transferring the executable image at migration time, which means the image must already be resident on the destination — and therefore on every node that is a candidate destination. Applying that to the entire task set would reserve memory on every node for every task, violating Requirement R6 on ECU-class hardware. Conversely, a mechanism that transfers the image on demand imposes a cost that depends on E_i , on S_i , and on the state of the network, and is therefore difficult to bound tightly.

The design resolves this by stratifying the mechanism by criticality. Three strategies are defined; they are described in full in Chapter 6 and summarised here only insofar as the architecture depends on them.

The HI strategy keeps a task resident in a standby state on a subset of the nodes, with only one instance executing at any time and runtime state held in a memory region shared between the candidate nodes and accessible only to tasks of the corresponding criticality. Migration then reduces to a stop signal on the source and a start signal on the destination, with no transfer of the executable and no transfer of state. The cost is correspondingly small and, more importantly, has small variance, which is what makes it amenable to the analysis Requirement R4 demands at the highest criticality level. The price is the memory reserved on every standby node and the shared-memory infrastructure itself, which is why the strategy is confined to the tasks that need it. For tasks requiring redundancy rather than availability — ASIL D functions with a duplicated-execution requirement — the same arrangement supports executing several instances concurrently and comparing their results.

The LO strategy transfers the executable image and the checkpoint over the network at migration time, allowing a task to be started on any admissible node without prior reservation of resources there. The cost depends on $E_i + S_i$ and on the network, and is characterised probabilistically rather than by a tight bound. The strategy makes efficient use of memory across the system and is applicable to the large majority of the task set.

The virtual-node strategy is a variant of the LO strategy targeting nodes hosted on MEC infrastructure. It differs in that the destination must first be instantiated — the master node requests the creation of a container — and in that the link latency δ_j is neither bounded nor under the control of the vehicle. Both differences are reasons the criticality ceiling of virtual nodes lies below the highest level.

Intermediate criticality levels are addressed by combining these mechanisms rather than by introducing further ones. Two combinations are available. A task may be held in standby, as under the HI strategy, but coordinated over the ordinary network rather than through shared memory, trading determinism for a lower memory footprint. Alternatively, the standby set may be reduced: in a system with two high-criticality sublevels, tasks at the upper sublevel are held in standby on every node while those at the lower sublevel are held on only a few, guaranteeing a prepared destination for the former while consuming fewer resources for the latter. Within the lower criticality levels, differentiation is expressed as transmission priority — tasks at higher sublevels are transferred first when several migrations are pending. These combinations are described conceptually and are not part of the realisation evaluated in Chapter ??.

4.6 Design Decisions

Several decisions shape the design above. They are collected here with the alternatives that were considered and the criterion that decided each.

Asymmetric rather than symmetric multiprocessing. Multi-core ECUs may be operated with a single scheduler across all cores (SMP) or with an independent instance per core (AMP). AMP is used here for two reasons. A software failure is confined to the core on which it occurs and affects only that core's task set, whereas under SMP a failure in shared scheduler state affects the whole device. And AMP makes each core an independent sub-node in the sense of Section 4.3.1, so that intra-ECU and inter-ECU migration are instances of one mechanism rather than two. The cost is that load cannot be balanced across cores by the scheduler itself; it must instead be balanced by the partitioning component, which is consistent with the architecture in any case.

FreeRTOS as the base kernel. The platform requires an operating system small enough for ECU-class hardware, open enough to be extended with dynamic loading and a mixed-criticality scheduler, and portable enough to run both on bare metal and inside a container. FreeRTOS satisfies all three, and its minimal footprint yields fast device startup, which matters for UC1 and UC2 where nodes are returned from standby. The alternatives considered were RTEMS, which offers a richer feature set at a larger footprint, and a hypervisor-based partitioning approach, which provides stronger isolation but assumes a static allocation of partitions at design time and imposes an

overhead disproportionate to the resources of the target devices. What FreeRTOS lacks — dynamic loading, cross-platform abstraction, state preservation, criticality-aware scheduling — is precisely what Chapter 5 adds.

Position-independent executables rather than process snapshots. Preserving a task across migration may be achieved by capturing its complete machine state, or by having the task declare the state it needs. The former is transparent to the application but requires capturing the program counter, stack and heap, and reproducing them on the destination; the transferred state is large and the machinery is substantial. The latter, adopted here, restricts migration to job boundaries (Assumption A3) and places a burden on the application developer (Assumption A4), but reduces the transferred state to a few kilobytes and requires no kernel-level snapshot support.

Containers rather than virtual machines for virtual nodes. Virtual nodes must present the same execution environment as physical nodes so that one binary serves both. A container hosting the same operating system achieves this with startup latency low enough for the on-demand instantiation UC3 and UC4 require, whereas a full virtual machine imposes a startup cost and a resource footprint that the transient availability of MEC infrastructure does not justify.

4.7 Scope of the Safety Argument

The motivation for this work is drawn from the automotive domain, and the criticality levels of Section 4.3.3 follow ISO 26262. It is therefore necessary to state precisely what is and is not claimed.

What is claimed is that the design is compatible with the requirements that functional safety standards impose on the *temporal* behaviour of safety-related functions. Specifically: tasks are scheduled by a mixed-criticality policy under which the guarantees of higher-criticality tasks do not depend on the behaviour of lower-criticality ones; migration events are subject to an admission test that preserves those guarantees; the cost of migration under the HI strategy is bounded and analysable; and nodes whose timing properties cannot be bounded are excluded from hosting high-criticality tasks by construction, through the criticality ceiling.

What is not claimed is any of the following. No component of the system is certified, and no certification argument is constructed (Assumption A10). Dynamic loading of executable code at runtime is in tension with the assumptions underlying current certification practice, and resolving that tension is an open question rather than a contribution of this work. Spatial freedom from interference is provided only to the extent that the memory protection facilities of the target hardware are employed, which Chapter 5 describes; the guest task model does not provide address space separation between tasks on one node. The transport of executable images is unauthenticated

Table 4.2: Traceability from research questions and requirements to mechanisms and chapters.

RQ	Req.	Mechanism	Chapter
RQ1	R1, R4	Job-boundary migration; migration cost model	6
RQ1	R2	Cooperative checkpointing; guest task library	5, 6
RQ2	R3	Mixed-criticality scheduler; criticality ceilings	5
RQ2	R3, R4	Admission of migrated tasks; mode-change interaction	6
RQ3	R3, R6	HI strategy: hot standby with shared state	6
RQ3	R2, R6	LO strategy: ELF and checkpoint transfer	6
RQ3	R5	Position-independent loading; service abstraction	5
RQ4	R5, R4	Virtual node strategy; criticality ceiling on MEC nodes	6
—	R4	PTP synchronisation and deadline compensation	??

(Assumption A9). And the in-vehicle network provides no deterministic latency bound (Assumption A8), so the timing results of Chapter ?? hold relative to an empirically measured δ_j rather than an analytically derived one.

These limitations are consequences of scoping decisions rather than of the design itself, and Section ?? identifies for each the work required to remove it.

4.8 Design Realisation and Status

The design described in this chapter is realised across Chapters 5 to ?? . Table 4.2 relates the research questions and requirements to the mechanisms that address them and to the chapter in which each is described.

Not every mechanism has reached the same stage of realisation. Table 4.3 states the status of each explicitly, so that the descriptions in the following chapters can be read as design descriptions without repeated qualification. Where a mechanism is designed but not yet evaluated, the corresponding chapter describes the design and states the evaluation that remains outstanding.

Table 4.3: Realisation status of the mechanisms at the time of writing.

Mechanism	Status	Evidence
Dynamic task loading (ELF)	implemented, evaluated	Ch. ??
Cross-platform execution	implemented, evaluated	Ch. ??
Cooperative checkpointing	implemented, evaluated	Ch. ??
LO migration strategy	implemented, evaluated	Ch. ??
Virtual node (MEC) strategy	implemented, partially evaluated	Ch. ??
PTP synchronisation	implemented, evaluated	Ch. ??
AMP multi-core deployment	implemented	Ch. 5
Mixed-criticality scheduler	partially implemented	Ch. 5
HI migration strategy	designed, evaluation outstanding	Ch. 6
MC-aware migration planning	designed, integration outstanding	Ch. ??

5 Base Operating System: Axcan OS

The architecture developed in Chapter 4 requires an execution platform on every node. That platform must satisfy Requirement R5 — the same task binary must run on physical and virtual nodes alike — and provide the mechanisms on which Requirements R1 to R3 depend: it must be able to receive and start a task it was not compiled with, to preserve and restore that task’s state, and to schedule tasks of differing criticality without allowing the less critical to interfere with the more critical. It must do so within a footprint appropriate to ECU-class hardware (Requirement R6).

This chapter describes Axcan OS, the platform developed to meet these requirements.¹ Section 5.1 states its relationship to the software it builds on. Section 5.2 presents the architecture. Sections 5.3 to 5.6 describe the four mechanisms that distinguish Axcan OS from its base: the mixed-criticality scheduler, dynamic task deployment, cross-platform task loading, and the guest task model. Sections 5.7 to 5.9 address memory management and isolation, failure handling, and resource footprint.

5.1 Design Rationale and Relationship to Existing Software

The choice of FreeRTOS as the base kernel was argued in Section 4.6; the argument is not repeated here. What matters for this chapter is the gap between what FreeRTOS provides and what the architecture requires.

FreeRTOS provides deterministic scheduling, task and queue primitives, and ports to a wide range of hardware, at a footprint of a few tens of kilobytes. It does not provide the following. Applications are compiled into the kernel image, so a node cannot execute a task it did not ship with. Scheduling is fixed-priority, with no priority-assignment policy and no notion of criticality. There is no mechanism for preserving task state across anything. And although the kernel itself is portable, the services an application needs — console output, networking, memory allocation — are reached through platform-specific interfaces, so an application compiled against one port will not run on another.

Axcan OS closes these four gaps. Its relationship to the software it builds on is as follows:

- **FreeRTOS** supplies the kernel: threading, context switching, queues, and the hardware ports used for both the embedded and the POSIX targets. It is used unmodified except where noted in Section 5.7.

¹The name derives from the Nahuatl *axcan*, meaning *now* or *immediately*.

- **ESFREE [esfree]** supplies the scheduling framework. ESFREE replaces the fixed-priority scheduler of FreeRTOS with a policy layer supporting static and dynamic real-time policies, among them rate-monotonic, deadline-monotonic and Earliest Deadline First (EDF). Axcan OS extends this layer with the mixed-criticality policies of Section 5.3.
- **Axcan OS** adds the deployment subsystem, the ELF loader and the cross-platform service abstraction, the guest task model and checkpoint mechanism, the mixed-criticality scheduling policies, and the AMP multi-core deployment described in Section 5.4.1.

A note on terminology is necessary. Axcan OS is often described as following a microkernel design. This is inaccurate and is not claimed here: FreeRTOS executes in a single address space with no privilege separation between kernel and application, and Axcan OS does not change this. What Axcan OS does adopt from microkernel systems is the *delegation of services*: functionality that an application would ordinarily reach through a library or a system call is instead provided by a component outside the application, reached through a uniform interface, and resolved at load time rather than at compile time. This is what makes a single binary portable across platforms (Section 5.5); it is not an isolation mechanism, and the isolation properties the platform does and does not provide are stated in Section 5.7.

5.2 Architecture

Figure 5.1 shows the components of Axcan OS and their interactions. The FreeRTOS kernel and the hardware abstraction layer form the foundation, providing deterministic scheduling and hardware interfacing. Above them, five components implement the behaviour the architecture requires.

Deployment communication. Maintains the connection to the master node, receiving task assignments, executable images and checkpoints, and reporting node health and per-task runtime status. On a multi-core node only the master core runs this component; the remaining sub-nodes run an inter-core variant (Section 5.4.1).

Deployment manager. Controls the task lifecycle: loading an executable into memory, instantiating it, starting and stopping it, registering it with the scheduler, monitoring its execution, and preparing its state for transfer when a migration event is issued.

Wrapper task. The kernel-level thread within which a dynamically loaded application executes. It mediates every interaction between the application and the rest of the system: it owns the application's memory, presents the service interface, invokes the application once per job, and performs checkpoint save and restore around that invocation (Section 5.6).

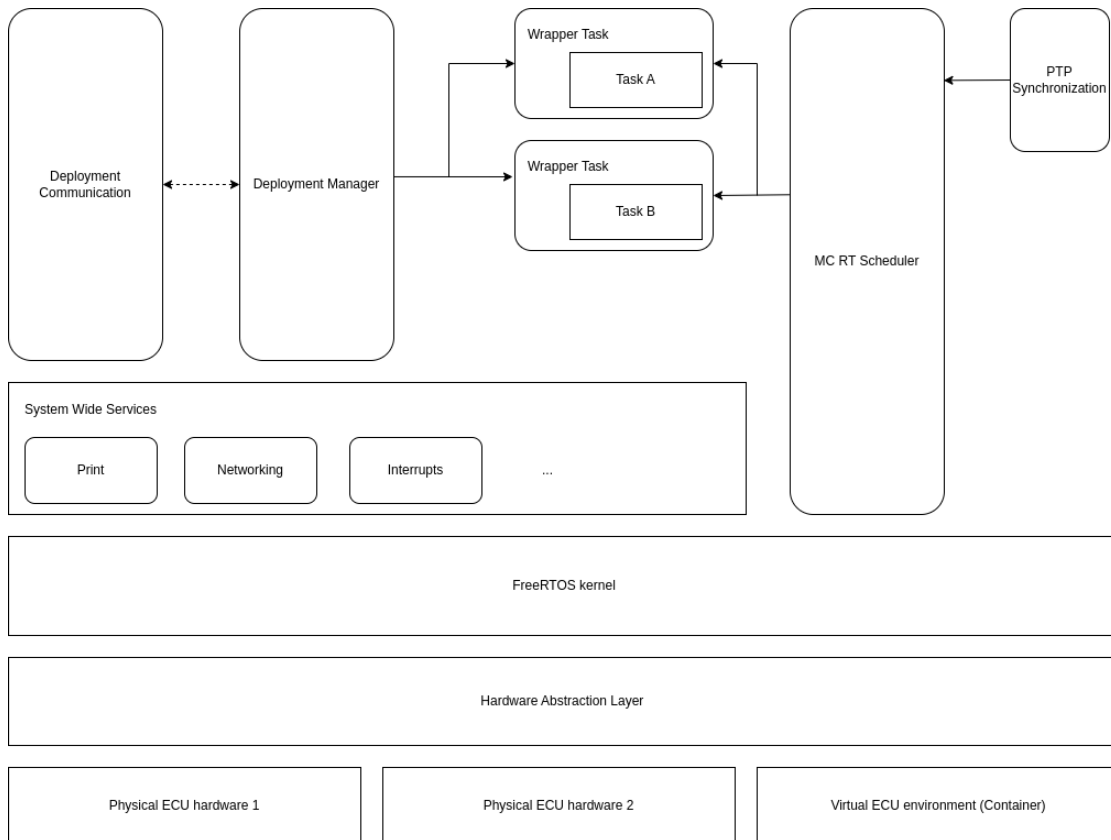


Figure 5.1: Axcan OS architecture. The FreeRTOS kernel and hardware abstraction layer are used as provided; the remaining components are contributions of this work.

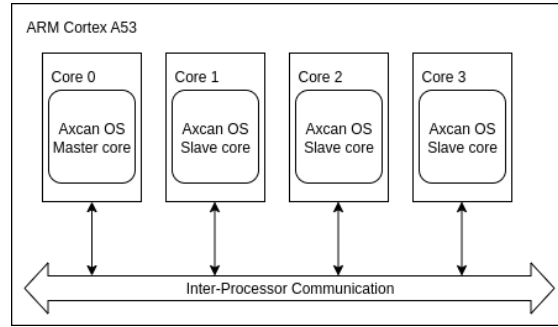


Figure 5.2: AMP deployment of Axcan OS on the quad-core ARM Cortex-A53 of the target hardware. Each core runs an independent instance and constitutes a node.

Scheduler. Selects which task executes, according to a configurable policy. Axcan OS supports both criticality-agnostic policies (rate-monotonic, deadline-monotonic, EDF) and the mixed-criticality policies of Section 5.3.

System services. Provides applications with a minimal API for functionality unavailable to a statically linked, dynamically loaded binary: console output, networking, memory allocation, and access to peripherals. Services are exposed as function pointers resolved at load time, which is what permits one binary to run on platforms whose underlying implementations differ.

Time synchronisation via PTP, shown in the figure, is a system-wide concern rather than a component of the node platform; the scheduler’s use of it is described in Section 5.3.5 and the protocol implementation in Chapter ??.

Together these components allow any sub-node — on a physical or a virtual node, single-core or one core of a multi-core ECU — to execute a task it was not compiled with, under real-time constraints, and to participate in migration.

5.2.1 Multi-Core Nodes

A multi-core ECU runs one instance of Axcan OS per core in an asymmetric multi-processing (AMP) arrangement, as illustrated in Figure 5.2 for the quad-core ARM Cortex-A53 of the target hardware. Each instance schedules its own task set independently and is therefore a sub-node in the sense of Section 4.3.1; tasks are assigned to sub-nodes, while the ECU as a whole is the node.

The instances are not quite identical. One core, conventionally core 0, is designated the *master core*. It is the node’s single point of contact towards the master node, it partitions the task set assigned to the node across its sub-nodes, and it owns those system services that require exclusive control of a hardware resource. The remaining *slave cores* reach those services through an inter-core wrapper interface, and reach the master node through the master core. Figure 5.3 shows the resulting differences.

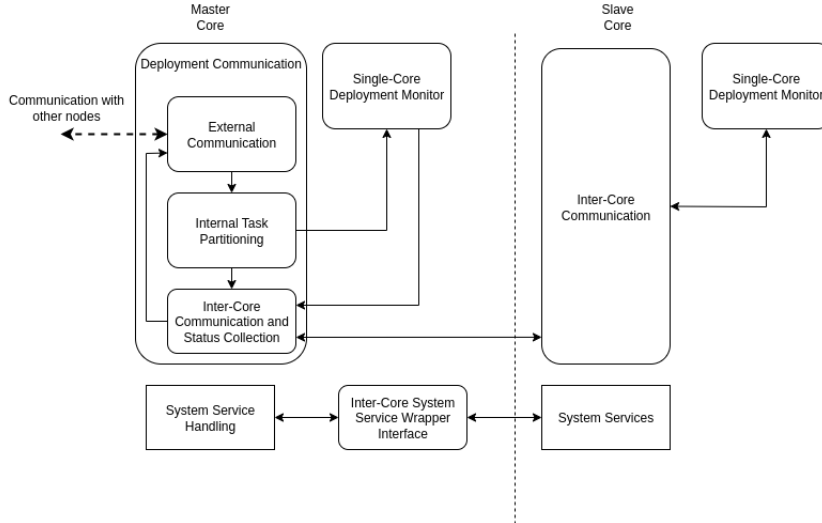


Figure 5.3: Differences between the Axcan OS instances on the master core and on a slave core.

5.3 Mixed-Criticality Scheduler

Requirement R3 demands that the guarantees given to a task of high criticality do not depend on the behaviour of tasks of lower criticality. On a single node this is a scheduling problem, and it is the problem this section addresses. Its interaction with migration — what happens when a task arrives at a node mid-hyperperiod, and how a mode change interacts with a migration in progress — is deferred to Section ??.

5.3.1 Baseline: EDF under ESFREE

The scheduler builds on the policy layer of ESFREE, which replaces the fixed-priority scheduler of FreeRTOS with one in which priorities are assigned by a configurable policy. The baseline policy is EDF, which is optimal for preemptive independent tasks on a uniprocessor in the sense that it schedules any task set that is schedulable at all [dertouzos1974; LL73]. Optimality does not survive the introduction of criticality: EDF makes no distinction between a deadline miss by a high-criticality task and one by a low-criticality task, and under overload it degrades unpredictably.

5.3.2 EDF with Virtual Deadlines

The mixed-criticality policy implemented in Axcan OS is EDF with virtual deadlines (EDF-VD) [baruah_edfvd]. The system operates in one of the criticality modes of $\mathcal{L}$, beginning in the lowest. In the low mode, every task is scheduled; tasks of high criticality are scheduled against a *virtual* deadline shorter than their true deadline, which reserves enough slack that they can still meet their true deadlines if the system

later switches to the high mode. If a job of a high-criticality task executes for longer than its low-mode budget $C_i(\text{LO})$, the system switches to the high mode: low-criticality tasks are suspended or degraded, and high-criticality tasks revert to their true deadlines with the larger budget $C_i(\text{HI})$.

The virtual deadline of a high-criticality task is obtained by scaling its true deadline by a factor $x \in (0, 1]$ common to all such tasks,

$$\hat{D}_i = x \cdot D_i, \quad (5.1)$$

with x chosen so that the task set is schedulable in both modes.

Note that the deadline used by the scheduler is the virtual deadline of Equation 5.1 applied to the compensated deadline D'_i of Equation 4.6, not to the global deadline D_i . The two adjustments compose: clock offset and link latency are corrected first, at the node level and uniformly across its task set, and the virtual deadline scaling is applied afterwards, at the task level and only to tasks of high criticality.

Mode changes. The mode-change protocol determines what happens to low-criticality tasks on a switch to the high mode, and under what condition the system returns to the low mode. Both choices affect the guarantees the system can offer and must be stated explicitly.

5.3.3 Realisation Variants

Two realisations of mixed-criticality scheduling on multi-core ECUs were developed.

Reserved cores. Each sub-node is assigned a criticality ceiling $L_k^{\max}$ (Section 4.3.3) and runs plain EDF over tasks at or below that ceiling. Criticality is thus enforced by partitioning rather than by the scheduling policy: because no core hosts tasks of differing criticality, no criticality-induced interference is possible within a core, and the per-core analysis reduces to ordinary EDF. Sub-nodes reserved for low-criticality work additionally serve as standby hosts for high-criticality tasks under the HI migration strategy (Section ??). The cost is reduced utilisation: capacity on a high-criticality core cannot be reclaimed by low-criticality work.

Hierarchical virtual-deadline EDF. A single core hosts tasks of several criticalities under EDF-VD, with a hierarchical server structure separating the criticality classes. This permits higher utilisation, at the cost of a more complex analysis and a mode-change protocol that affects all tasks on the core.

5.3.4 Admission Control

A node admits a task only if Equation 4.4 holds, whose third conjunct is the local schedulability test. For the reserved-core variant this is the EDF utilisation bound over

the tasks resident on the core; for the EDF-VD variant it is the two-mode condition of Section 5.3.2.

5.3.5 Timing Basis

The scheduler's notion of time is the node-local clock, synchronised to the system-wide reference by PTP and bounded in deviation by Δ_j . The sub-nodes of a node share that clock. Deadlines arrive from the master node expressed globally and are converted on receipt according to Equation 4.6. Because the correction is uniform across the task sets of all sub-nodes of a node, it preserves the relative ordering of deadlines and therefore does not alter the sequence of scheduling decisions; it does, however, consume slack on every task resident on the node. The implementation of the protocol itself, and the measured behaviour of Δ_j , are described in Chapter ?? and Chapter ?? respectively.

5.4 Dynamic Task Deployment

A node must be able to execute a task set that is decided at runtime. Two threads implement this.

The *deployment communication* thread maintains the connection to the master node. It reports node health and, per task, the number of jobs completed, the number of deadlines missed, and the time of the last execution. In the other direction it receives the assigned task set, the executable images and checkpoints required to start those tasks, and instructions to start, stop or transfer a task. Received instructions are placed in a shared memory block from which the deployment manager consumes them.

The *deployment manager* executes those instructions. Starting a task comprises: receiving the executable image and checkpoint; allocating and initialising the memory the task requires; loading the image (Section 5.5); creating the wrapper task that will host it; and registering the task with the scheduler, which thereafter controls when it runs. A monitoring sub-thread then checks periodically that the task is executing correctly and collects the runtime statistics reported by the communication thread. The monitor is also what stops or pauses a task on instruction, and what prepares and triggers the transfer of a task's files when a migration event requires it.

5.4.1 Multi-Core Deployment

On a multi-core node the master core receives the task set assigned to the device as a whole and partitions it across the sub-nodes. The partitioning uses a greedy algorithm that balances load while accounting for the additional management load carried by the master core itself.

Having partitioned, the master core sends each slave core its share over the inter-core channel and starts its own share through its deployment manager. Slave cores replace the deployment communication thread with an inter-core variant that speaks to the

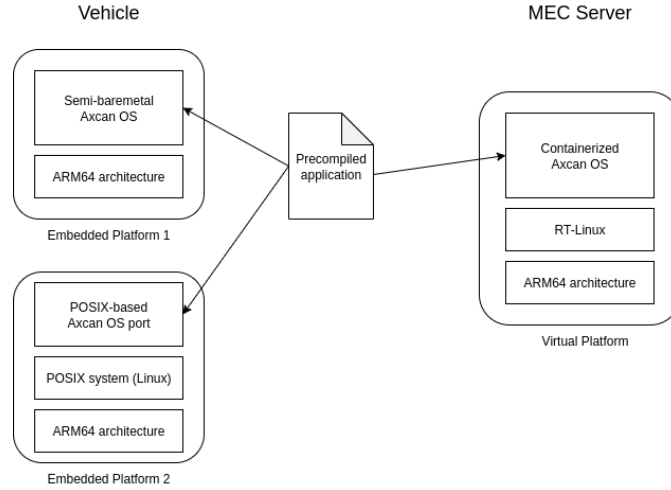


Figure 5.4: A single precompiled application executing across a bare-metal embedded platform, a POSIX-hosted platform, and a containerised virtual node, all sharing the ARM AArch64 architecture.

master core rather than to the master node; everything below that substitution — the deployment manager, the monitor, the wrapper tasks, the scheduler — is identical on every core. Status reporting is aggregated in the opposite direction: the master core collects its own status and that of the slave cores and emits a single node-level report to the master node.

5.4.2 Inter-Core Communication

Cores communicate through ring buffers in a shared memory region, signalled by inter-processor interrupts. On this substrate a message-queue abstraction is provided by the *post office* library [horst; wiesboeck], which gives each core a receiving thread that accepts messages into a queue for the corresponding management thread to process.

Communication between any pair of cores is possible, although in this deployment nearly all traffic is between the master core and a slave core. The exception, and the reason the general case is supported, is the shared state region used by the HI migration strategy (Section ??).

5.5 Cross-Platform Task Loading

Requirement R5 states that one binary must serve every admissible node. Axcan OS achieves this with an ELF loader together with a service abstraction; this section describes both, and Figure 5.4 shows the platforms across which a single binary has been demonstrated to execute.

The loader accepts an application compiled as a position-independent executable, allocates memory for it, performs the necessary relocations, and transfers control to its entry point. No recompilation is required for different targets, provided an Axcn OS port runs on the target and the architecture matches the one the application was compiled for (Assumption A5). The loader was implemented in large part in collaboration with Yinbo Zhou [zhou_thesis].

5.5.1 Hardware Abstraction Layer

Portability of the kernel is not sufficient for portability of applications. The official FreeRTOS ports abstract the hardware from the kernel, but the services an application uses — console output, networking, memory allocation, peripheral access — remain reachable only through platform-specific interfaces. Axcn OS therefore adds an abstraction layer of its own, with two parts.

The first is the encapsulation of system calls behind uniform signatures. Each service is exposed to the application through a function pointer with a platform-independent signature, resolved at load time to whichever implementation the host platform provides. Because resolution occurs at load time rather than link time, the application binary carries no dependency on any particular platform.

The second concerns memory allocation for the loaded image, where the platforms differ in a way that cannot be hidden behind a signature. On a bare-metal port, memory may be allocated without regard to the host, though alignment must respect the target and manual management is required where the port provides no memory management unit. On a port hosted over POSIX — the containerised virtual node, or a development build on a UNIX host — the underlying operating system governs memory, and memory obtained through `malloc` is not, in general, executable. Allocation there uses `mmap` with explicit protection flags, which is more flexible but requires the loader to manage the mapping explicitly. The loader selects between the two at build time of the port, not at load time of the application, so this difference is invisible to the application.

5.6 The Guest Task Model

An application that is to be loaded dynamically, executed under a mixed-criticality scheduler, and migrated between nodes cannot be an arbitrary program. This section states the contract it must satisfy and the support Axcn OS provides. The contract has three parts: how the application is compiled, how it is structured, and how it declares its state.

5.6.1 The Wrapper Task

A loaded application does not execute as a kernel thread of its own. It executes inside a *wrapper task*: a FreeRTOS thread created by the deployment manager, one

per application, which mediates everything between the application and the system. The wrapper owns the memory into which the image was loaded; it holds the service function pointers and the resource table and passes them to the application at entry; it invokes the application once per job rather than letting it run freely; and it performs checkpoint restore before that invocation and checkpoint save after it.

Concentrating these responsibilities in the wrapper is what allows the application itself to remain an ordinary, statically linked, position-independent program with no knowledge of the platform, the scheduler, or migration. It is also what makes a job the unit of scheduling and of migration: because the wrapper, not the application, controls invocation, the system has a well-defined point at which the application holds no state outside its declared structure — which is precisely the quiescent point Assumption A3 relies on.

5.6.2 Compilation Requirements

An application must be compiled as a position-independent executable (PIE), and without the standard C library startup code and system call layer that would bind it to a host. It links instead against a minimal standalone C library. It must also include the Axcan OS headers, which provide the entry point handling, the service interface, the resource table, and the checkpoint mechanism described below.

5.6.3 Execution Model and Checkpoints

Applications are periodic and follow the sense–think–act pattern: the application’s main function constitutes a single job, which reads its inputs, computes, and produces outputs. Any state that must survive from one job to the next is loaded at the start of the main function and stored back at its end. This is the minimal form of the checkpoint mechanism, and under it a migrated task resumes at a job boundary.

Applications with long-running jobs may declare additional checkpoints within the body of a job. Each such checkpoint stores the state together with a jump tag; at the start of the following invocation the tag directs control to the corresponding point in the code, so that the job resumes from the most recent completed stage rather than from the beginning. Both forms are shown in Figure 5.5.

The state itself is declared by the application as a structure and registered in a *resource table*, through which the wrapper serialises and restores it without knowledge of its contents. The correctness of migration therefore depends on the application declaring its state completely (Assumption A4); the platform can enforce the mechanism but not the completeness.

The serialised form of the checkpoint, its metadata and integrity check, and its use during a migration event are described in Section ?? . This section describes only the mechanism by which state is captured; its transfer is a matter for Chapter 6.

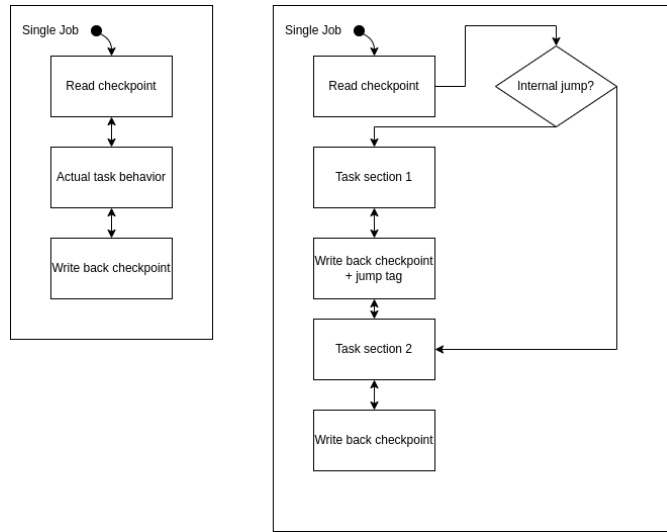


Figure 5.5: Guest application structure. Left: the minimal form, in which state is restored at the start of a job and stored at its end. Right: the extended form with an intra-job checkpoint and jump tag, permitting resumption from an internal point.

5.7 Memory Management and Isolation

Section 4.7 states that spatial freedom from interference is provided only to the extent that the memory protection facilities of the target hardware are used. This section states what that extent is.

5.8 Failure Handling

Dynamic loading introduces failure modes that a statically linked system does not have. Axcen OS handles them as follows.

The last two rows connect to the scheduler: an overrun of the high-mode budget is the trigger for the mode change of Section 5.3.2, and an unmet service requirement is a violation of the admissibility condition of Equation 4.4 that should have been prevented by the master node and is therefore reported as such.

5.9 Resource Footprint

Requirement R6 constrains the platform to hardware of the class the architecture targets. Table 5.2 reports the cost of the additions Axcen OS makes to its base.

Per-task cost should be reported alongside: the wrapper task's own stack and control structures, and the memory reserved for a task's image and state, are recurring costs that determine how many tasks a node can host and, under the HI strategy, how many

Table 5.1: Failure modes introduced by dynamic loading and their handling.

Failure	Detection	Response
Malformed or truncated ELF image	—	—
Unsupported relocation type	—	—
Insufficient memory for the image	—	—
Checkpoint fails its integrity check	—	—
Checkpoint size or version mismatch	—	—
Application faults during a job	—	—
Application overruns $C_i(HI)$	—	—
Required service unavailable on node	—	—

Table 5.2: Static memory footprint of Axcan OS relative to the base kernel, ARM Cortex-A53 port.

Component	Text (KiB)	Data + BSS (KiB)
FreeRTOS kernel (baseline)	—	—
ESFREE scheduling layer	—	—
Mixed-criticality policies	—	—
Deployment subsystem	—	—
ELF loader	—	—
Service abstraction	—	—
Inter-core communication	—	—
Total	—	—

standby copies the system can afford.

5.10 Summary

This chapter described Axcan OS, the execution platform on which every node of the breathable system runs. Built on the FreeRTOS kernel and the ESFREE scheduling layer, it adds four mechanisms: a mixed-criticality scheduler that prevents low-criticality tasks from interfering with the timing of high-criticality ones; a deployment subsystem through which a node receives and manages a task set decided at runtime; an ELF loader and service abstraction that allow one binary to execute on bare-metal, POSIX-hosted and containerised nodes alike; and a guest task model whose wrapper task and checkpoint mechanism give migration a well-defined quiescent point and a well-defined

unit of state.

Together these satisfy the platform-level obligations of Requirements R1 to R3 and R5: a node can start a task it was not compiled with, preserve and restore that task's state, schedule it according to its criticality, and do so identically regardless of what the node is. What remains — how a task is moved between two such nodes, at what cost, and under what conditions the move preserves timing guarantees — is the subject of Chapter 6.

The realisation status of the mechanisms described here is stated in Table 4.3. Axcan OS is a proof-of-concept platform: the set of system services offered to applications is minimal, the isolation properties are those of Section 5.7, and the inter-node communication mechanisms admit more efficient realisations than the ones implemented.

6 Task Migration Implementation

6.1 Statefulness and Checkpoint Mechanism

In order to keep the running state of the system tasks during migration (i.e. making it stateful), a checkpoint mechanism should be implemented. In this context, checkpoint refers to a data file containing all relevant variables to store or reload a snapshot of the task's state, whereas mechanism refers to the whole process of storing and reloading the task. There is a trade-off between the precision of the checkpoint mechanism and its simplicity and amount of checkpoint data. For example, a task's state could be very precisely restored down to a specific instruction if stuff like program counter and full task context (e.g. stack and heap, and nested data structure references) can be snapshotted. Another advantage of this strategy is that tasks could be implemented without being specific to the system, but the drawback is that it might require keeping a lot of data and a more complicated implementation at kernel level. On the other hand, checkpoints can be defined in the code at some strategic steps, along with the most relevant processed variables (e.g. for an object detection algorithm after every processing step of an image). This might mean that some work could be lost after migration, but the checkpoint structure would be simple and the mechanism would involve only some code jumps, although the task would need to be specifically implemented to work with such a mechanism. The approach for the checkpoint mechanism in this work is closer to the latter, and the reasoning behind this design decision, as well as the details of its implementation are described next.

First of all, let us recall the nature of the system and the executed tasks. As mentioned before, these tasks are all assumed to be periodic, in each iteration (job) needing to process some information, possibly based on sensors or other peripherals, and generating an output, that might also depend on previous iterations. Given this behavior, it is assumed that at least, a checkpoint can be stored after a full iteration while keeping a meaningful state. Additionally, other internal checkpoints could be made by keeping jump tags if higher level of granularity is needed. During development phase, the checkpoint has to be designed in the form of a task structure which can contain all the relevant data and be stored in a file. At runtime, task reloading requires three fundamental steps, as shown in diagram...: 1) reading the existing checkpoint to get previous state, 2) performing the actual job, and 3) writing back the new state in a checkpoint. Additionally, if jump tags are stored, after retrieving the previous state, the corresponding jump is performed, while it is also important to save the checkpoint before the next sub step of the job (i.e. next jump tag), including a hint to the jump

tag. The design decision to follow this strategy was taken considering the smaller overhead at runtime and during implementation of the OS. Also, since aforementioned steps 1 and 3 (reading and writing back the state) would be the same for most simple applications, they can be pre-implemented via a library, making it only necessary to define the task state data structure, and otherwise keeping the application code fairly simple. Furthermore, flexibility for fine granularity in the checkpoint keeping is possible through the mentioned jump tags and by allowing to override the specific behavior of steps 1 and 3.

An example of the above mentioned behavior could be a SLAM (simultaneous localization and mapping) algorithm, which builds a cumulative map of the surroundings based on the received sensor data and finds the current position of the exploring agent (e.g. a vehicle) in that map. In this case, results of previous jobs (the generated map and the agent's position) are also used to generate the results of the current iteration. For this task, a minimal state could be stored by keeping track of those results, but as it is a more complex task with several steps, jump tags could be kept for example after each of the relevant steps: pre-processing sensor data, sensor fusion, identifying agent's position, filling the map with new data. This second behavior allows for a finer granularity for migrating the task while still avoiding the overhead of a full kernel-space snapshot.

While the previous paragraphs define the general behavior of the checkpoint mechanism, it is also necessary to define the way in which it is transmitted from the device currently executing the task (source) to the device that will execute next (target). As mentioned before, the user should define a data structure with all relevant data inside the task code, this will be then accessible through a void pointer from the OS in binary format, which would be further processed to share the checkpoint with the migration target. In general, defining the data structure in this way allows for a variety of implementations, as long as the data stored in it keeps its integrity, and the specific transmission approach will depend on the specific migration strategy according to the criticality of the task and the type of target device as described in the next sections.

6.2 High Criticality Strategy

*** MISSING details now, but idea is explained in approach***

- The idea is based on a premise that a shared memory approach would allow faster and less data-intensive transmission for performing the migration
- The task has 3 possible states on each device: off, running and (hot) standby. For tasks that do not require redundancy at runtime, only one (active) device (node in the distributed system) keeps them in running state, while a selection of backup nodes keep it in standby mode. To enable runtime redundancy, multiple nodes

can be active. Nodes that are neither active nor selected as backup keep task in an off state.

- Both running and hot-standby modes keep the task loaded on the system and ready to execute, but standby mode does not make task perform any work, while running mode does.
- State is kept through a shared memory approach. In the scope of this work (so far), the concept is explored using multiple cores in a single multicore processor as if they were independent nodes, with the processor memory working as the shared memory.
- Conceptually, this memory can represent a distributed shared memory (DSM), which could be further integrated using a real-time communication protocol to keep time-bound behavior. Example of this approach might be... (Do proper referencing...) <https://doi-org.eaccess.tum.edu/10.1007/s10586-017-1140-9> <https://dl.acm.org/doi/abs/10.1007/107-1140-9>
- The idea would be to keep a fixed reference to this space in memory for all running and standby instances, which would be read before and updated after each executed job.

6.3 Low Criticality Strategy

- Based greatly on Yinbo's thesis
- Here, idea is that tasks can be precompiled to ELF files which can be loaded dynamically upon request
- Task data are stored in a separate file that can be loaded with the ELF and later stored upon request
- Explain how tasks must be compiled and prepared to provide valid ELF files.
- Loader would work with any scheduling algorithm, ensuring that MC can be respected in the system

6.4 MEC: Virtual Devices

- Based greatly on Roberto's thesis
- Here, idea is that the MEC can host containerized versions of the devices – Implementation specific details for the containers are out of scope in the thesis, as they are part of Bernhard's idea

6 Task Migration Implementation

- Explain briefly that it is assumed the implementation of the containerized version of the tasks is fully implemented in the scope of future work
- Explain how the migration to and from the cloud works
- Explain how the migration to and from the MEC is integrated into the system architecture, especially how it is considered when taking a decision to migrate to MEC or to another physical device
- Discuss missing implementation details

The work presented in (cite ROberto's thesis) showed the possibility of containerizing a POSIX port of FreeRTOS and running it on a Linux server. Following this concept, Axcan OS was ported to make it compatible with such a container approach, making it possible to load on the virtual devices exactly the same ELF executables that are loaded for the real embedded devices, avoiding the need for recompiling the tasks. Furthermore, the features of Axcan OS allow the tasks to access the system services in a seamless manner.

The container-compatible POSIX port of Axcan OS requires a different management of memory and peripherals, especially due to the restrictions introduced by the Linux kernel, for example due to the exception level or the rights assigned to memory allocated with functions like malloc or mmap. However, Axcan OS offers an abstraction layer that offers the features to the user applications on the same way on both platforms.

7 System Integration

7.1 Real-Time Constraints in the Distributed System

7.2 Adaption of the ML algorithm

7.3 Virtual and Physical Devices

8 Results and Evaluation

8.1 Test Setup

8.2 Discussion

9 Conclusion & Outlook

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Bibliography

- [Aut] *AUTOSAR Adaptive Platform: Main Requirements, Release 22-11*. Tech. rep. Available at <https://www.autosar.org>. AUTOSAR Consortium, 2023.
- [Bak91] T. P. Baker. "Stack-Based Scheduling of Real-Time Processes." In: *Real-Time Systems* 3.1 (1991), pp. 67–99. doi: 10.1007/BF00365393.
- [Bar+11] S. Baruah, V. Bonifaci, G. D'Angelo, A. Marchetti-Spaccamela, S. van der Ster, and L. Stougie. "Preemptive Uniprocessor Scheduling of Mixed-Criticality Sporadic Task Systems." In: *Proceedings of the 23rd Euromicro Conference on Real-Time Systems (ECRTS)*. 2011, pp. 145–154. doi: 10.1109/ECRTS.2011.20.
- [Bau+18] M. Baunach, R. Martins Gomes, M. Malenko, F. Mauroner, L. Batista Ribeiro, and T. Scheipel. "Smart mobility of the future - a challenge for embedded automotive systems." In: *e & i Elektrotechnik und Informationstechnik*. Vol. 135. 4. 2018, pp. 304–308. doi: 10.1007/s00502-018-0623-6.
- [Bau+18] M. Baunach, R. M. Gomes, M. Malenko, F. Mauroner, L. B. Ribeiro, and T. Scheipel. "Smart Mobility of the Future – A Challenge for Embedded Automotive Systems." In: *e & i Elektrotechnik und Informationstechnik* 135.4 (2018), pp. 304–308.
- [BB13] A. Burns and S. Baruah. "Sustainability in Real-Time Scheduling." In: *Journal of Computing Science and Engineering* 7.4 (2013), pp. 235–247. doi: 10.5626/JCSE.2013.7.4.235.
- [BD17] A. Burns and R. I. Davis. "A Survey of Research into Mixed Criticality Systems." In: *ACM Computing Surveys* 50.6 (2017).
- [Ben+23] S. Bensalem et al. "High-Performance Computing Platforms for Autonomous Vehicles: Architectures and Challenges." In: *IEEE Access* 11 (2023), pp. 108233–108260.
- [Bli19] B. Blieninger. *Online Machine Learning for improved decision making regarding the migration of automotive software components at runtime*. 2019.
- [BLK22] J. Baik, J. Lee, and K. Kang. "Task Migration and Scheduler for Mixed-Criticality Systems." In: *Sensors* 22.5 (2022), p. 1926. doi: 10.3390/s22051926.
- [Bur+18] O. Burkacky, J. Deichmann, G. Doll, and C. Knochenauer. *Rethinking Car Software and Electronics Architecture*. Tech. rep. McKinsey & Company, Feb. 2018.

Bibliography

- [But11] G. C. Buttazzo. *Hard Real-Time Computing Systems*. 3rd ed. Springer, 2011.
- [CL12] Y.-Y. Chen and C.-M. Lyu. "ECU-Level Fault-Tolerant Framework for Safety-Critical FlexRay Network Systems." In: *International Conference on ICT Convergence*. 2012, pp. 553–558.
- [CM94] C. Cardeira and Z. Mammeri. "Neural Network-Based Approach for Real-Time Scheduling." In: *Proceedings of the IEEE Workshop on Neural Networks for Signal Processing*. 1994, pp. 576–585. doi: 10.1109/NNSP.1994.366059.
- [Cra+16] S. S. Craciunas, R. S. Oliver, M. Chmelik, and W. Steiner. "Scheduling Real-Time Communication in IEEE 802.1Qbv Time Sensitive Networks." In: *Proceedings of the 24th International Conference on Real-Time Networks and Systems (RTNS)*. 2016, pp. 183–192. doi: 10.1145/2997465.2997470.
- [DB11] R. I. Davis and A. Burns. "A Survey of Hard Real-Time Scheduling for Multiprocessor Systems." In: *ACM Computing Surveys* 43.4 (2011), 35:1–35:44. doi: 10.1145/1978802.1978814.
- [Del21] O. I. Delgadillo Ruiz. *Hardware-in-the-Loop Test Setup for a Generalistic Approach to Machine-Learning-Based Schedulability Analysis*. 2021.
- [Do1] DO-178C: *Software Considerations in Airborne Systems and Equipment Certification*. Tech. rep. RTCA DO-178C / EUROCAE ED-12C. RTCA / EUROCAE, 2011.
- [Dür+19] M. Dürr, D. Ziegenbein, A. Hamann, and R. Ernst. "End-To-End Timing Analysis of Sporadic Cause-Effect Chains in Distributed Systems." In: *ACM Transactions on Embedded Computing Systems* 18.5s (2019), 58:1–58:24. doi: 10.1145/3358181.
- [DW04] J. Dominguez and A. Wellings. "Applying Neural Networks to Fixed-Priority Scheduling of Sporadic Task Sets." In: *Proceedings of the 10th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2004. doi: 10.1109/RTAS.2004.1247674.
- [EKB15] S. Eckl, D. Krefft, and U. Baumgarten. "KIA4SM - Cooperative Integration Architecture for Future Smart Mobility Solutions." In: *Conference on Future Automotive Technology (CoFAT)*. Vol. 4. Apr. 2015.
- [Ela+24] M. Elamin et al. "Software-Defined Vehicle Architecture: A Survey and Research Directions." In: *IEEE Access* 12 (2024), pp. 42115–42148.
- [Eln+02] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson. "A Survey of Rollback-Recovery Protocols in Message-Passing Systems." In: *ACM Computing Surveys* 34.3 (2002), pp. 375–408. doi: 10.1145/568522.568525.
- [Eng+03] J. Engblom, A. Ermedahl, M. Sjödin, J. Gustafsson, and H. Hansson. "Worst-Case Execution-Time Analysis for Embedded Real-Time Systems." In: vol. 4. 4. 2003, pp. 437–455. doi: 10.1007/s10009-002-0080-7.

- [FPY13] L. L. Ferreira, L. M. Pinho, and P. M. Yomsi. "Task Migration for Fault-Tolerance in Mixed-Criticality Embedded Systems." In: *Work-in-Progress Proceedings of the Euromicro Conference on Real-Time Systems (ECRTS)*. DFT4FTT project. 2013.
- [HC16] C. Hainz and A. Chauhan. *Automotive Change Drivers for the Next Decade*. Tech. rep. EY Global Automotive & Transportation Sector, 2016.
- [Hei20] G. Heiser. "Hypervisors for Consumer Electronics." In: *Proceedings of the IEEE Consumer Electronics Show Conference*. 2020. DOI: 10.1109/ICCE46568.2020.9042985.
- [HGB+24] D. Hoornaert, G. Ghaemi, A. Bastoni, et al. "MCTI: Mixed-Criticality Task-Based Isolation." In: *Real-Time Systems* 60 (2024), pp. 328–365.
- [Ieea] *IEEE Std 802.1AS: Timing and Synchronization for Time-Sensitive Applications*. Tech. rep. IEEE 802.1AS-2020. IEEE, 2020.
- [Ieeb] *IEEE Std 802.1Q: Bridges and Bridged Networks*. Tech. rep. IEEE 802.1Q-2022. IEEE, 2022.
- [Iso] *ISO 26262: Road Vehicles—Functional Safety, Parts 1–12*. Tech. rep. Second Edition. International Organization for Standardization, 2018.
- [Jia+22] X. Jiang, T. Sha, D. Liu, J. Chen, C. Chen, and K. Huang. "Flexible and Dynamic Scheduling of Mixed-Criticality Systems." In: *Sensors* 22.19 (2022), p. 7528. DOI: 10.3390/s22197528.
- [JP86] M. Joseph and P. Pandya. "Finding Response Times in a Real-Time System." In: *The Computer Journal* 29.5 (1986), pp. 390–395. DOI: 10.1093/comjnl/29.5.390.
- [KSK18] F. Kuhnert, C. Stürmer, and A. Koster. *Five Trends transforming the Automotive Industry*. Tech. rep. PricewaterhouseCoopers GmbH Wirtschaftsprüfungsgesellschaft, 2018.
- [LL73] C. L. Liu and J. W. Layland. "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment." In: *Journal of the ACM* 20.1 (1973), pp. 46–61.
- [Ma+22] Y. Ma, W. Liu, T. Gong, Y. Jiang, M. Xu, and K. G. Shin. "A Survey of Energy-Aware Scheduling in Mixed-Criticality Systems." In: *Journal of Systems Architecture* 126 (2022), p. 102476. DOI: 10.1016/j.sysarc.2022.102476.
- [Man+13] R. Mancuso, R. Pellizzoni, M. Caccamo, L. Sha, and H. Yun. "WCET (m) Estimation in Multi-Core Systems Using Single Core Equivalence." In: *Proceedings of the 25th Euromicro Conference on Real-Time Systems (ECRTS)*. 2013, pp. 174–183. DOI: 10.1109/ECRTS.2013.27.

Bibliography

- [Mao+17] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief. "A Survey on Mobile Edge Computing: The Communication Perspective." In: *IEEE Communications Surveys & Tutorials* 19.4 (2017), pp. 2322–2358. doi: 10.1109/COMST.2017.2745201.
- [Mec] *Mobile Edge Computing (MEC): Framework and Reference Architecture*. Tech. rep. GS MEC 003. V3.1.1. ETSI, 2022.
- [Mel+17] A. Melani, M. Bertogna, V. Bonifaci, A. Marchetti-Spaccamela, and G. C. Buttazzo. "Response-Time Analysis of Conditional DAG Tasks in Multiprocessor Systems." In: *Proceedings of the 29th Euromicro Conference on Real-Time Systems (ECRTS)*. 2017, pp. 1–22. doi: 10.4230/LIPIcs.ECRTS.2017.1.
- [MKA16] Y. Maruyama, S. Kato, and T. Azumi. "Exploring the Performance of ROS2." In: *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*. 2016. doi: 10.1145/2968478.2968502.
- [Mor17] R. Morabito. "Virtualization on Internet of Things Edge Devices with Container Technologies: A Performance Evaluation." In: *IEEE Access* 5 (2017), pp. 8835–8850. doi: 10.1109/ACCESS.2017.2704444.
- [MSD10] T. Megel, R. Sirdey, and V. David. "Minimizing Task Preemptions and Migrations in Multiprocessor Optimal Real-Time Schedules." In: *31st IEEE Real-Time Systems Symposium*. 2010, pp. 37–46.
- [Omg] *Data Distribution Service (DDS) Specification, Version 1.4*. Tech. rep. OMG Document formal/2015-04-10. Object Management Group (OMG), 2015.
- [Ose] *OSEK/VDX Operating System Specification 2.2.3*. Tech. rep. Available at <https://www.osek-vdx.org>. OSEK Group, 2005.
- [RGF22] F. Reghenzani, Z. Guo, and W. Fornaciari. "A Mixed-Criticality Approach to Fault Tolerance: Integrating Responses to Faults with Mode Changes." In: *IEEE Transactions on Computers* 71.11 (2022), pp. 2917–2930. doi: 10.1109/TC.2022.3144113.
- [Sal+23] M. Salehi et al. "Adaptive Mixed-Criticality Scheduling for Energy-Aware Real-Time Systems." In: *Journal of Systems Architecture* 138 (2023), p. 102867.
- [St22] T. Stüber, L. Osswald, S. Lindner, and M. Menth. "A Survey of Scheduling Algorithms for the Time-Aware Shaper in Time-Sensitive Networking (TSN)." In: *arXiv preprint arXiv:2211.10954* (2022). Published version: *IEEE Communications Surveys & Tutorials*.
- [Tai19] S. Taieb. *Deep Learning Based Schedulability Analysis for Migration of Software Components at Runtime*. 2019.
- [Tos23] A. Tosić. "Run-Time Application Migration Using Checkpoint/Restore in Userspace." In: *arXiv preprint arXiv:2307.12113* (2023). Empirical study of CRIU-based container migration for edge computing scenarios.

- [TV24] E. Tinto and T. Vardanega. "Providing Spatial Isolation for Mixed-Criticality Systems." In: *Journal of Systems Architecture* 154 (2024), p. 103234.
- [Utz19] T. Utz. *Vergleich von heuristischen und Deep Learning basierten Schedulability Analyse Verfahren für die Migration von Softwarekomponenten zur Laufzeit*. 2019.
- [Ves07] S. Vestal. "Preemptive Scheduling of Multi-Criticality Systems with Varying Degrees of Execution Time Assurance." In: *Proceedings of the 28th IEEE International Real-Time Systems Symposium*. 2007, pp. 239–243.
- [Vip+14] K. Vipin, S. Shreejith, S. A. Fahmy, and A. Easwaran. "Mapping Time-Critical Safety-Critical Cyber Physical Systems to Hybrid FPGAs." In: *2014 IEEE International Conference on Cyber-Physical Systems, Networks, and Applications*. 2014, pp. 31–36. doi: 10.1109/CPSNA.2014.14.
- [Vip18] K. Vipin. "CANNoC: An open-source NoC architecture for ECU consolidation." In: *2018 IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS)*. 2018, pp. 940–943. doi: 10.1109/MWSCAS.2018.8624006.
- [Wil+08] R. Wilhelm et al. "The Worst-Case Execution-Time Problem—Overview of Methods and Survey of Tools." In: *ACM Transactions on Embedded Computing Systems* 7.3 (2008), 36:1–36:53. doi: 10.1145/1347375.1347389.
- [XZZ+24] C. Xue, T. Zhang, Y. Zhou, et al. "Real-Time Scheduling for IEEE 802.1Qbv Time-Sensitive Networking (TSN): A Systematic Review and Experimental Study." In: *ACM Computing Surveys* (2024). Published online ahead of print. doi: 10.1145/3736715.
- [You+17] C. You, K. Huang, H. Chae, and B.-H. Kim. "Energy-Efficient Resource Allocation for Mobile-Edge Computation Offloading." In: *IEEE Transactions on Wireless Communications* 16.3 (2017), pp. 1397–1411. doi: 10.1109/TWC.2016.2633522.

A Additional Stuff

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.